

Práctica 4

Análisis de algoritmos

1 Ejercicio 1

Debido a un error en la actualización de sus sistemas, el banco AyED perdió la información del estado de todas sus cuentas. Afortunadamente logran recuperar un backup del día anterior y utilizando las transacciones registradas en las últimas 24hrs podrán reconstruir los saldos. Hay poco tiempo que perder, el sistema bancario debe volver a operar lo antes posible. Las transacciones se encuentran en consultas, una consulta cuenta con un valor y un rango de cuentas consecutivas a las que hay que aplicar este cambio, por ejemplo la consulta $(333..688 = 120)$ implica sumar \$120 a todas las cuentas entre la número 333 y la número 688 (inclusive). Entonces, la recuperación de los datos consiste en aplicar todas las consultas sobre el estado de las cuentas recuperadas en el backup del día anterior. El equipo de desarrollo se pone manos a la obra y llega a una solución rápidamente (Algoritmo **procesarMovimientos**). Toman cada consulta y recorren el rango de cuentas aplicando el valor correspondiente, como muestra el siguiente algoritmo.

```
Consultas.comenzar();
while(!consultas.fin()){
    Consultas = consultas.proximas();
    for (i= consulta.desde; i < consulta.hasta; i++){
        cuenta[i] = cuenta[i] + consulta.valor;
    }
}
```

Escriben la solución en pocos minutos y ponen en marcha el proceso de recuperación. Enseguida se dan cuenta que el proceso va a tardar muchas horas en finalizar, son muchas cuentas y muchos movimientos, la solución aunque simple es ineficiente. Luego de discutir varias ideas llegan a una solución (Algoritmo **procesarMovimientosOptimizado**) que logra procesar toda la información en pocos segundos. Ambos algoritmos se encuentran en el archivo Ejercicio 1 - rsq.tn.ayed.zip del material adicional.

a.- Para que usted pueda experimentar el tiempo que demora cada uno de los dos algoritmos en forma empírica, usted debe ejecutar cada uno de ellos, con distintas cantidades de elementos y completar la tabla. Luego haga la gráfica para comparar los tiempos de ambos algoritmos. Tenga en cuenta que el algoritmo posee dos constantes CANTIDAD_CUENTAS y CANTIDAD_CONSULTAS, sin embargo, por simplicidad, ambas toman el mismo valor. Sólo necesita modificar CANTIDAD_CUENTAS.

N° Cuentas (y consultas)	procesarMovimientos	procesarMovimientosOptimizado
1.000	0,039	0,002
10.000	0,545	0,009
25.000	4,044	0,021
50.000	15,552	0,024
100.000	60,337	0,034

b.- ¿Por qué procesarMovimientos es tan ineficiente? Tenga en cuenta que pueden existir millones de movimientos diarios que abarquen gran parte de las cuentas bancarias.

Es ineficiente porque por cada consulta recorre uno por uno todos los índices de su rango para sumar el valor. El costo de una consulta depende del tamaño del rango, no es constante. Con millones de movimientos que abarcan gran parte de las cuentas, el trabajo total es del orden de $n \times m$ (cuentas $\times$ consultas), un algoritmo cuadrático, $O(n^2)$.

c.- ¿En qué se diferencia procesarMovimientosOptimizado? Observe las operaciones que se realizan para cada consulta.

Por cada consulta hace solo 2 operaciones, sin importar el tamaño del rango: suma el valor en `aux[desde]` y resta ese mismo valor en `aux[hasta+1]`. Eso es $O(1)$ por consulta $\rightarrow O(m)$ en total. Después, un único recorrido final del arreglo, acumulando (`aux[i] += aux[i-1]`), propaga el efecto de cada consulta a lo largo de su rango y lo "corta" justo donde termina. Ese paso es $O(n)$.

2 Ejercicio 2

La clase `BuscadorEnArrayOrdenado` del material adicional (Ejercicio 2 - Tiempo.zip) resuelve el problema de buscar un elemento dentro de un array ordenado. El mismo problema, lo resuelve de dos maneras diferentes: búsqueda lineal y búsqueda dicotómica. Se define la variable `cantidadElementos`, la cual se va modificando para determinar una escala (por ejemplo de a 100.000 o 1.000.000, dependiendo de la capacidad de cada equipo). Realice una tabla con el tiempo que tardan en ejecutarse ambos algoritmos, para los distintos valores de la variable.

N	Lineal	Dicotómica
100.000	2	0
200.000	3	0
300.000	2	0
400.000	3	0

3 Ejercicio 3

En la documentación de la clase `arrayList` que se encuentra en el siguiente link <https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html>

Se encuentran las siguientes afirmaciones

- "The size, isEmpty, get, set, iterator, and listIterator operations run in constant time."
- "All of the other operations run in linear time (roughly speaking)"

Explique con sus palabras por qué cree que algunas operaciones se ejecutan en tiempo constante y otras en tiempo lineal.

- Algunas operaciones son $O(1)$ porque `ArrayList` guarda los elementos en un arreglo de tamaño fijo, que se va agrandando cuando hace falta. Un arreglo permite acceso directo con el índice de la posición, la memoria calcula directamente la dirección donde está ese elemento. No se recorre nada.
- Por otro lado, todo lo que implica buscar un elemento sin conocer su posición o modificar la estructura del arreglo, requiere recorrerlo. Por ejemplo, en `contains(x)` hay que recorrer y comparar la lista elemento a elemento.

A partir del siguiente ejercicio, tener en cuenta la siguiente tabla:

Ordenadas en forma creciente	Nombre
1	Constante
log n	Logaritmo
n	Lineal
n log n	n Log n
n²	Cuadrática
n³	Cúbica
cⁿ c>1	Exponencial

4 Ejercicio 4

Determinar si las siguientes sentencias son verdaderas o falsas, justificando la respuesta utilizando notación Big-Oh.

- a. 3^n es de $O(2^n)$

La afirmación es falsa. Para demostrarlo, hallamos un c_1 que acote el término 2^n .

$$\begin{aligned}3^n &\leq 2^n \\ \log_2(3^n) &\leq n \\ n \cdot \log_2 3 &\leq n \\ \log_2 3 &\leq 1\end{aligned}$$

Como $\log_2 3 \approx 1.58$, es mayor a 1. Lo que obtuvimos es un absurdo, es decir, no existe c_1 que acote el término.

- b. $n + \log_2(n)$ es de $O(n)$

La afirmación es verdadera. Para demostrarlo, analizamos término a término los valores de n_0 para los que vale la desigualdad.

- Primer término: n .
En este caso, $c_1 = 1$.

$$\begin{aligned}n &\leq n \\ 1 &\leq 1 \\ 1 &= 1\end{aligned}$$

Con $c_1 = 1$, el término se puede acotar con $n_0 = 1$

- Segundo término: $\log_2(n)$
En este caso, $c_2 = 1$. Hallamos el n_0 que cumple la desigualdad.

$$\begin{aligned}\log_2(n) &\leq n \\ \log_2(1) &\leq 1 \\ 0 &\leq 1\end{aligned}$$

Con $c_2 = 1$, el término se puede acotar con $n_0 = 1$.

Ahora, hay que obtener el c y el n_0 que permita acotar la función. Para esto, juntamos los resultados parciales que obtuvimos antes, sumando las desigualdades (sumamos todos los términos del lado derecho y todos los del lado izquierdo).

Tener en cuenta que los resultados anteriores son:

- $T(n) = n + \log_2(n)$
- $c_1 = 1$
- $c_2 = 1$

$$n + \log_2(n) \leq c_1 n + c_2 n$$

$$T(n) \leq (c_1 + c_2)n$$

$$T(n) \leq (1 + 1)n$$

$$T(n) \leq 2n$$

Llamamos $c=2$.

$$T(n) \leq cn$$

La constante c es la que se obtiene de sumar todas las c_i correspondiente a cada término i . Ahora elegimos el n_0 más restrictivo, el que cumple que todos los términos, que en este caso nos queda que se cumple para $n \geq 1$. Por lo tanto,

$$T(n) \leq O(n), \text{ con } c=2 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$

c. $n^{1/2} + 10^{20}$ es de $O(n^{1/2})$.

La afirmación es verdadera. Para demostrarlo, analizamos término a término los valores de n_0 para los que vale la desigualdad y los c_i que acoten los términos.

- Primer término: $n^{1/2}$
Para este término $c_1 = 1$.

$$\begin{aligned} n^{1/2} &\leq n^{1/2} \\ (n^{1/2})^2 &\leq n \\ 1 &\leq n \end{aligned}$$

La desigualdad vale para todo $n_0 \geq 1$, con $c_1 = 1$.

- Segundo término: 10^{20}
Despejamos c_1 .

$$\begin{aligned} 10^{20} &\leq n^{1/2} \\ (10^{20})^2 &\leq n \\ 10^{40} &\leq n \end{aligned}$$

Ahora, busquemos el valor de c y el n_0 que nos permita acotar la función.

$$n^{1/2} + 10^{20} \leq c_1 n^{1/2} + c_2 n^{1/2}$$

$$T(n) \leq (c_1 + c_2)n^{1/2}$$

$$T(n) \leq (1 + 1)n^{1/2}$$

$$T(n) \leq 2n^{1/2}$$

Llamamos $c=2$.

$$T(n) \leq cn^{1/2}$$

La constante c es la que se obtiene de sumar todas las c_i correspondiente a cada término i . Ahora elegimos el n_0 más restrictivo, el que cumple que todos los términos, que en este caso nos queda que se cumple para $n \geq 10^{40}$. Por lo tanto,

$$T(n) \leq O(n^{1/2}), \text{ con } c=2 \text{ para todo } n \geq n_0, \text{ con } n_0 = 10^{40}.$$

d.

$$\begin{cases} 3n + 17 & n < 100 \\ 317 & n \geq 100 \end{cases}$$

La afirmación es verdadera. Para demostrarlo, analizamos cada trozo de la función y a su vez los términos de cada una de ellas.

(a) $3n + 17$ para $n < 100$

- Primer término $3n$.

Consideramos

$$n \leq n$$

Si multiplicamos ambos lados de la desigualdad por 3. Para mantener la desigualdad y para obtener, del lado izquierdo, el primer término de la función $T(n)$.

$$3n \leq 3n$$

Del lado izquierdo tenemos el primer término del trozo de la función que estamos analizando y del otro tenemos un valor c_1 que acota la función. Tenemos $c_1 = 3$ para $n_0 \geq 0$.

- Segundo término: 17

Tomamos $c_2 = 1$

$$17 \leq n$$

La desigualdad se cumple para $n_0 \geq 17$.

Obtenemos el valor de c y n_0 con el que podemos acotar la función. Tenemos de un lado la función y la suma de los resultados parciales del otro. Recordar:

- $c_1 = 3$
- $c_2 = 1$

$$3n + 17 \leq c_1n + c_2n$$

$$T(n) \leq (c_1 + c_2)n$$

$$T(n) \leq (3 + 1)n$$

$$T(n) \leq 4n$$

Nos queda entonces $c=4$ y el n_0 más restrictivos que cumple en todos los términos, es decir, $n_0 = 17$. Por lo tanto,

$$T(n) \leq O(n), \text{ con } c=4 \forall n, n < 100, \text{ con } n_0 = 17.$$

(b) 317 para $n \geq 100$

- Término: 317
Si tomamos $c_3 = 317$

$$317 \leq c_3 n$$

$$317 \leq 317n$$

Tenemos al valor $c_3 = 317$ como valor que acota al término. Siendo que, por la definición de la función, este término se cumple para los $n_0 \geq 100$, podemos tomar $n_0 = 100$.

- e. Mostrar que $p(n) = 3n^5 + 8n^4 + 2n + 1$ es de $O(n^5)$. Verdadero. Hay que demostrar que $p(n)$ es $O(n^5)$. Para esto, hay que hallar la constante $c > 0$ y n_0 tales que:

$$3n^5 + 8n^4 + 2n + 1 \leq cn^5 \text{ para todo } n \geq n_0$$

Analizamos que la desigualdad se cumpla en todos los términos.

- Primero término: $3n^5$.
Comenzamos con

$$n^5 \leq n^5$$

Ahora, multiplicamos por 3 de ambos lados.

$$3n^5 \leq 3n^5$$

Del lado izquierdo tenemos el primer término de $p(n)$, mientras que del lado derecho obtuvimos un valor para $c_1, c_1 = 3$. Si $n = 1$,

$$\begin{aligned} 3(1)^5 &\leq 3(1)^5 \\ 3 &\leq 3 \end{aligned}$$

La desigualdad es cierta. El primer término se puede acotar con $c_1 = 3$ y $n_0 = 1$, vale para $n_0 \geq 1$.

- Segundo término: $8n^4$
Comenzamos con

$$n^4 \leq n^5$$

Multiplicamos por 8 en ambos lados.

$$8n^4 \leq 8n^5$$

Del lado izquierdo tenemos el segundo término de $p(n)$, mientras que del lado derecho obtuvimos un valor para $c_2, c_2 = 8$. Si $n = 1$,

$$\begin{aligned} 8(1)^4 &\leq 8(1)^5 \\ 8 &\leq 8 \end{aligned}$$

La desigualdad es cierta. El segundo término de la función se puede acotar con $c_2 = 8$ y $n_0 = 1$, vale para $n_0 \geq 1$.

- Tercer término: $2n$
Comenzamos con

$$n^2 \leq n^5$$

Multiplicamos por 2 en ambos lados.

$$2n \leq 2n^5$$

Del lado izquierdo tenemos el tercer término de $p(n)$, mientras que del lado derecho obtuvimos un valor para $c_3, c_3 = 2$. Si $n = 1$,

$$\begin{aligned} 2(1) &\leq 2(1)^5 \\ 2 &\leq 2 \end{aligned}$$

La desigualdad es cierta. El tercer término de la función p se puede acotar con $c_3 = 2$ y $n_0 = 1$, vale para $n_0 \geq 1$.

- Cuarto término: 1
Comenzamos con

$$1 \leq n^5$$

Del lado izquierdo tenemos el cuarto término de $p(n)$, mientras que del lado derecho obtuvimos un valor para $c_4, c_4 = 1$. Si $n = 1$,

$$\begin{aligned} 1 &\leq 1(1)^5 \\ 1 &\leq 1 \end{aligned}$$

La desigualdad es cierta. El primer término se puede acotar con $c_4 = 1$ y $n_0 = 1$, vale para $n_0 \geq 1$.

Ahora, necesitamos obtener el c y n_0 que permite acotar la función $p(n)$, vamos a juntar todos los resultados parciales que obtuvimos. Recordar:

- $c_1 = 3$
- $c_2 = 8$
- $c_3 = 2$
- $c_4 = 1$

La constante c será la resultante de sumar todos los c_i obtenidos anteriormente. Sumamos todas las desigualdades:

$$\begin{aligned} 3n^5 + 8n^4 + 2n + 1 &\leq c_1n^5 + c_2n^5 + c_3n^5 + c_4n^5 \\ p(n) &\leq (c_1 + c_2 + c_3 + c_4)n^5 \\ p(n) &\leq (3 + 8 + 2 + 1)n^5 \\ p(n) &\leq 14n^5 \end{aligned}$$

Para obtener el n_0 final, tomamos el n_0 más restrictivo entre los de los resultados parciales, el que cumple con todas las desigualdades, es decir, $n_0 = 1$. Por lo tanto,

$$p(n) \leq O(n^5), \text{ con } c=14 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$

- f. Si $p(n)$ es un polinomio de grado k , entonces $p(n)$ es $O(n^k)$.

Para este caso, hay que tener en cuenta que si $p(n)$ es un polinomio de grado k , entonces

$$p(n) = c_1n^k + c_2n^{k-1} + c_3n^{k-2} + \dots + c_{k-1}n + c_k$$

Como queremos demostrar que es $O(n^k)$, deberíamos analizar la función término a término pero como no sabemos cuántos tiene la función, lo vamos a hacer para algunos de manera general.

- Primer término: c_1n^k .
Consideramos

$$n^k \leq n^k$$

Si multiplicamos por c_1 a cada lado.

$$c_1n^k \leq c_1n^k$$

La desigualdad se mantiene, c_1 acota ese término de la función. Tomando $n_0 = 1$:

$$\begin{aligned} c_1(1)^k &\leq c_1(1)^k \\ c_1 &\leq c_1 \end{aligned}$$

La desigualdad es verdadera, entonces se cumple para $n_0 \geq 1$. El término se acota con c_1 y $n_0 = 1$.

- Segundo término: c_1n^{k-1} .
Consideramos

$$n^{k-1} \leq n^k$$

Si multiplicamos por c_2 a cada lado.

$$c_2 n^{k-1} \leq c_2 n^k$$

La desigualdad se mantiene, c_2 acota ese término de la función.
Tomando $n_0 = 1$:

$$\begin{aligned} c_2(1)^{k-1} &\leq c_2(1)^k \\ c_2 &\leq c_2 \end{aligned}$$

La desigualdad es verdadera, entonces se cumple para $n_0 \geq 1$. El término se acota con c_2 y $n_0 = 1$.

- Término k-1: $c_{k-1}n$
Consideramos

$$n \leq n^k$$

Si multiplicamos por c_{k-1} a cada lado.

$$c_{k-1}n \leq c_{k-1}n^k$$

La desigualdad se mantiene, c_{k-1} acota ese término de la función.
Tomando $n_0 = 1$:

$$\begin{aligned} c_{k-1}(1) &\leq c_{k-1}(1)^k \\ c_{k-1} &\leq c_{k-1} \end{aligned}$$

La desigualdad es verdadera, entonces se cumple para $n_0 \geq 1$. El término se acota con c_{k-1} y $n_0 = 1$.

- Término k: c_k
Consideramos

$$1 \leq n^k$$

Si multiplicamos por c_k a cada lado.

$$c_k \leq c_k n^k$$

La desigualdad se mantiene, c_k acota ese término de la función.
Tomando $n_0 = 1$:

$$\begin{aligned} c_k &\leq c_k(1)^k \\ c_k &\leq c_k \end{aligned}$$

La desigualdad es verdadera, entonces se cumple para $n_0 \geq 1$. El término se acota con c_k y $n_0 = 1$.

Ahora, juntamos todos los resultados parciales para obtener el c y n_0 que acoten a $p(n)$.

$$c_1 n^k + c_2 n^{k-1} + c_3 n^{k-2} + \dots + c_{k-1} n + c_k \leq c_1 n^k + c_2 n^k + \dots + c_{k-1} n^k + c_k n^k$$

El lado izquierdo de la desigualdad es la función $p(n)$ y le lado derecho los resultados que fuimos obteniendo antes. Reescribiendo el lado izquierdo y sacando factor común n^k del lado derecho.

$$p(n) \leq (c_1 + c_2 + \dots + c_{k-1} + c_k) n^k$$

Si renombramos $c = c_1 + c_2 + \dots + c_{k-1} + c_k$,

$$p(n) \leq c n^k$$

El valor de n_0 que hace que la desigualdad se cumpla para todos los términos es $n_0 = 1$ Por lo tanto,

$$p(n) \leq O(n^k), \text{ con un valor } c \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$

5 Ejercicio 5

Se necesita generar una permutación random de los n primeros número enteros. Por ejemplo $[4,3,1,0,2]$ es pero $[0,4,1,2,4]$ no lo es, porque un número está duplicado (el 4) y otro no está (el 3). Presentamos tres algoritmos para solucionar este problema. Asumimos la existencia de un generador de número random, $ran_int(i, j)$ el cual genera en tiempo constante, enteros entre i y j inclusive con igual probabilidad (esto significa que puede retornar el mismo valor más de una vez). También suponemos el mensaje $swap()$ que intercambia dos datos entre sí.

```
public class EjercicioPermutaciones {
    private static Random rand = new Random();

    public static int[] randomUno(int n) {
        int i, x = 0, k;
        int[] a = new int[n];
        for (i = 0; i < n; i++) {
            boolean seguirBuscando = true;
            while (seguirBuscando) {
                x = ran_int(0, n - 1);
                seguirBuscando = false;
                for (k = 0; k < i && !seguirBuscando; k++)
                    if (x == a[k])
                        seguirBuscando = true;
            }
        }
    }
}
```

```

        a[i] = x;
    }
    return a;
}

public static int[] randomDos(int n) {
    int i, x;
    int[] a = new int[n];
    boolean[] used = new boolean[n];
    for (i = 0; i < n; i++) used[i] = false;
    for (i = 0; i < n; i++) {
        x = ran_int(0, n - 1);
        while (used[x]) x = ran_int(0, n - 1);
        a[i] = x;
        used[x] = true;
    }
    return a;
}

public static int[] randomTres(int n) {
    int i;
    int[] a = new int[n];
    for (i = 0; i < n; i++) a[i] = i;
    for (i = 1; i < n; i++) swap(a, i, ran_int(0, i - 1));
    return a;
}

private static void swap(int[] a, int i, int j) {
    int aux;
    aux = a[i]; a[i] = a[j]; a[j] = aux;
}

/** Genera en tiempo constante, enteros entre i y j con igual probabilidad.
 */
private static int ran_int(int a, int b) {
    if (b < a || a < 0 || b < 0)
        throw new IllegalArgumentException("Parámetros inválidos");

    return a + (rand.nextInt(b - a + 1));
}

public static void main(String[] args) {
    System.out.println(Arrays.toString(randomUno(1000)));
    System.out.println(Arrays.toString(randomDos(1000)));
    System.out.println(Arrays.toString(randomTres(1000)));
}
}

```

- a. Analizar si todos los algoritmos terminan o alguno puede quedar en loop infinito.
- b. Describa con palabras la cantidad de operaciones que realizan.

a. randomUno

El algoritmo puede entrar en loop infinito porque *ran_int* puede generar número repetidos. Por ejemplo, cuando $i = 1$, *ran_int* puede retornar 0 o 1. Si retorna 0, sería un valor repetido de la iteración anterior ($i = 0$) y entraría en loop infinito.

randomDos

El algoritmo puede entrar en loop infinito porque existe la posibilidad de que *ran_int* genere números repetidos. Por ejemplo, en la segunda iteración ($i = 1$), puede generar el valor 0 todas las veces y nunca saldría del while.

randomTres

El algoritmo no entra en loop porque, con el método swap, se intercambian posiciones.

b. randomUno

Como en el peor de los casos se ejecuta infinitas veces, la cantidad de operaciones será infinita.

randomDos

Como en el peor de los casos se ejecuta infinitas veces, la cantidad de operaciones será infinita.

randomTres

Realiza una cantidad de operaciones fija y predecible, sin importar si es el mejor o el peor de los casos El primer for hace exactamente n asignaciones para llenar el arreglo con $[0, 1, 2, \dots, n - 1]$. El segundo for se ejecuta exactamente $n - 1$ veces. En cada iteración hace una llamada a *ran_int* y un swap (el cual consume 3 asignaciones internas).

6 Ejercicio 6

- a. Supongamos que tenemos un algoritmo de $O(\log^2 n)$ y disponemos de 1 hora de uso de CPU. En esa hora, la CPU puede ejecutar el algoritmo con una entrada de tamaño $n = 1024$ como máximo. ¿Cuál sería el mayor tamaño de entrada que podría ejecutar nuestro algoritmo si disponemos de 4 horas de CPU?

Repasemos los datos que tenemos:

- El orden de la función de $T(n)$ es de $O(\log^2 n)$.

- En 1 hora, $n = 1024$

Tengamos en cuenta que $\log^2 n$ es el orden de la función. Es decir, en la función, puede haber una constante c_0 que multiplique al logaritmo.

La forma aproximada de $T(n)$: $T(n) = c_0 \log^2 n$ Entonces,

$$1 = T(1024)$$

$$1 = c_0 \log^2 1024$$

Despejamos la constante c_0 .

$$c_0 = \frac{1}{\log^2 1024}$$

Ahora que tenemos el valor de la constante c_0 , la podemos reemplazar en la función $T(n)$.

$$T(n) = c_0 \log^2 n$$

$$T(n) = \frac{1}{\log^2 1024} \cdot \log^2 n$$

Ahora buscamos la cantidad de operaciones (el valor de n) para 4 horas.

$$T(n) = \frac{1}{\log^2 1024} \cdot \log^2 n$$

$$4 = \frac{1}{\log^2 1024} \cdot \log^2 n$$

$$4 \cdot \log^2 1024 = \log^2 n$$

$$\sqrt{4 \log^2 1024} = \sqrt{\log^2 n}$$

Aplico raíz para deshacerme del exponente

$$2 \log 1024 = \log n$$

$$n = 10^{2 \log 1024}$$

$$n = 1048576$$

- b. Considerando que un algoritmo requiere $T(n)$ operaciones para resolver un problema y la computadora procesa 10.000 operaciones por segundo. Si $T(n) = n^2$, determine el tiempo en segundos requerido por el algoritmo para resolver un problema de tamaño $n = 2.000$.

Repasemos los datos que tenemos:

- La cantidad de operaciones n para $T(n) = n^2$ en un segundo es de $T(n) = 10000$.
- Tenemos que hallar el tiempo que lleva resolver un problema de tamaño 2.000 para un algoritmo con una función $T(n) = n^2$.

Primero calculamos el valor de $T(n)$ para $n = 2.000$.

$$\begin{aligned}T(n) &= n^2 \\T(2000) &= 2000^2 \\T(2000) &= 4000000\end{aligned}$$

Ya tenemos cuánto tiempo lleva resolver 10000 operaciones y la queremos saber cuánto lleva resolver 4000000 de operaciones, por lo que hacemos una regla de tres simples:

$$\begin{array}{rcl}10000 \text{ operaciones} & \text{_____} & 1 \text{ segundo} \\4000000 \text{ operaciones} & \text{_____} & x = 400 \text{ segundos}\end{array}$$

Para resolver el problema se requieren 400 segundos.

7 Ejercicio 7

Para cada uno de los algoritmos presentados:

- Expresar en función de n el tiempo de ejecución.
- Establecer el orden de dicha función usando notación Big-Oh.

1)

```
for(int i = 0; i < n + 100; ++i) {
    for(int j = 0; j < i * n ; ++j)
        sum = sum + j;
    for(int k = 0; k < n + n + n; ++k)
        c[k] = c[k] + sum;
}
```

- En el código tenemos una iteración con otras dos iteraciones dentro. Cada for va a representar una sumatoria.

- Segundo for
Comienza desde $j = 0$ hasta $i \cdot n - 1$. Esto es porque el límite superior escrito en el for es estrictamente menor a $i \cdot n$ y el índice aumenta en uno en cada iteración. Hay dos opciones:
 - Tomar el límite inferior desde $j = 0$ hasta $i \cdot n - 1$.

- Tomar el límite inferior desde $j = 1$ hasta $i \cdot n$.

Voy a elegir la segunda opción. La sumatoria me queda:

$$\sum_{k=1}^{i \cdot n} c_1$$

- Tercer for

Comienza desde $k = 0$ hasta $n + n + n - 1$, que es igual a $3n - 1$. Esto es porque el límite superior escrito en el for es estrictamente menor a $3n$ y el índice aumenta en uno en cada iteración. Hay dos opciones:

- Tomar el límite inferior desde $k = 0$ hasta $3n - 1$.
- Tomar el límite inferior desde $k = 1$ hasta $3n$.

Voy a elegir la segunda opción. La sumatoria me queda

$$\sum_{k=1}^{3n} c_2$$

- Primer for

Comienza desde $i = 0$ hasta $n+99$. Esto es porque el límite superior escrito en el for es estrictamente menor a $n + 100$ y el índice aumenta en uno en cada iteración. Hay dos opciones:

- Tomar el límite inferior desde $i = 0$ hasta $n+99$.
- Tomar el límite inferior desde $i = 1$ hasta $n+100$.

Voy a elegir la segunda opción. Esta sumatoria está compuesta por la suma de las anteriores.

$$\sum_{i=1}^{n+100} \left(\sum_{k=1}^{i \cdot n} c_1 + \sum_{k=1}^{3n} c_2 \right)$$

Que en este caso, también corresponde a la "forma inicial" de $T(n)$.

Podemos armar una expresión de $T(n)$.

$$T(n) = \sum_{i=1}^{n+100} \left(\sum_{k=1}^{i \cdot n} c_1 + \sum_{k=1}^{3n} c_2 \right)$$

Ahora, tenemos que "deshacernos" de las sumatorias y hacer que todo quede en función de n .

Propiedades que voy a usar:

- $\sum_{i=1}^n c = (n - 1 + 1) \times c = n \times c$
- $\sum_{i=1}^n c \cdot a_i = c \times \sum_{i=1}^n a_i$

- $\sum_{i=1}^n a_i + b_i = \sum_{i=1}^n a_i + \sum_{i=1}^n b_i$
- $\sum_{i=1}^n i = \frac{n(n+1)}{2}$

Entonces,

$$\begin{aligned}
T(n) &= \sum_{i=1}^{n+100} \left(\sum_{j=1}^{i \cdot n} c_1 + \sum_{k=1}^{3n} c_2 \right) = \\
&= \sum_{i=1}^{n+100} (c_1 i \cdot n + c_2 3 \cdot n) = \\
&= \sum_{i=1}^{n+100} c_1 i n + \sum_{i=1}^{n+100} c_2 3n = \\
&= c_1 n \sum_{i=1}^{n+100} i + c_2 3n \sum_{i=1}^{n+100} 1 = \\
&= c_1 n \left(\frac{(n+100)(n+101)}{2} \right) + c_2 3n(n+100) = \\
&= \frac{c_1}{2} n(n^2 + 201n + 201) + c_2 3n^2 + c_2 300n \\
&= \frac{c_1}{2} n^3 + \frac{c_1}{2} 201n^2 + \frac{c_1}{2} 201n + c_2 3n^2 + c_2 300n = \\
&= \frac{c_1}{2} n^3 + \left(\frac{c_1}{2} 201 + c_2 3 \right) n^2 + \left(\frac{c_1}{2} 201 + c_2 300 \right) n
\end{aligned}$$

Llamando

- $c_3 = \frac{c_1}{2}$
- $c_4 = \frac{c_1}{2} 201 + 3c_2$
- $c_5 = 201 \frac{c_1}{2} + 300c_2$

Tenemos el tiempo de ejecución T en función de n.

$$T(n) = c_3 n^3 + c_4 n^2 + c_5 n$$

b. Para establecer el orden de la función, hay que hacer la demostración. Podemos decir que n^3 crece más rápido que n^2 y que n . Se tiene que demostrar que

$$T(n) = c_3 n^3 + c_4 n^2 + c_5 n \leq O(n^3)$$

Para esto, tenemos que encontrar una constante $k > 0$ y un n_0 tales que

$$c_3n^3 + c_4n^2 + c_5n \leq kn^3 \text{ para todo } n \geq n_0$$

Empezamos analizando término a término.

- Primer término: c_3n^3
Sabemos que $n^3 \leq n^3$ Si multiplicamos a ambos lados por c_3 ,

$$c_3n^3 \leq c_3n^3$$

Encontramos un valor $k_1 = c_3$. Ahora, si probamos con $n_0 = 0$.

$$c_3 \cdot 0^3 \leq c_3 \cdot 0^3$$

La desigualdad se cumple. El término se puede acotar con $k_1 = c_3$ y $n_0 = 0$, para $n_0 \geq 0$.

- Segundo término: c_4n^2
Sabemos que $n^2 \leq n^3$ Si multiplicamos a ambos lados por c_4 ,

$$c_4n^2 \leq c_4n^3$$

Encontramos un valor $k_2 = c_4$. Ahora, si probamos con $n_0 = 0$.

$$c_4 \cdot 0^2 \leq c_4 \cdot 0^3$$

La desigualdad se cumple. El término se puede acotar con $k_2 = c_4$ y $n_0 = 0$, para $n_0 \geq 0$.

- Tercer término: c_5n
Sabemos que $n \leq n^3$ Si multiplicamos a ambos lados por c_5 ,

$$c_5n \leq c_5n^3$$

Encontramos un valor $k_3 = c_5$. Ahora, si probamos con $n_0 = 0$.

$$c_5 \cdot 0 \leq c_5 \cdot 0^3$$

La desigualdad se cumple. El término se puede acotar con $k_3 = c_5$ y $n_0 = 0$, para $n_0 \geq 0$.

Ahora, sumamos todos los resultados parciales que obtuvimos antes.

$$c_3n^3 + c_4n^2 + c_5n \leq k_1n^3 + k_2n^3 + k_3n^3$$

$$T(n) \leq (k_1 + k_2 + k_3)n^3$$

$$T(n) \leq (c_3 + c_4 + c_5)n^3$$

Renombrando $c = c_3 + c_4 + c_5$

$$T(n) \leq cn^3$$

El n_0 más restrictivo es $n_0 = 0$ porque cumple con todos los términos para $n_0 \geq 0$. Por lo tanto,

$T(n) \leq O(n^3)$, con $c = c_3 + c_4 + c_5$ para todo $n \geq n_0$, con $n_0 = 0$.

2)

```
int i,j;
int x = 1;
for (i = 0; i <= n^2; i=i+2)
for (j = n; j >= 1; j-= n/4)
    x++;
```

a. Primero tenemos dos operaciones constantes, las líneas

```
int i,j;
int x = 1;
```

y dos iteraciones anidadas que corresponden a dos sumatorias, una externa y otra interna. Vamos a analizar como va cambiando el índice de cada una.

- Sumatoria interna.

Este for empieza con $j = n$ y en cada "vuelta" se le resta $\frac{n}{4}$ al valor de j hasta que sea 1. Entonces

1. $j = n$ (primera vuelta)
2. $j = n - \frac{n}{4}$ (segunda vuelta)
3. $j = n - \frac{n}{4} - \frac{n}{4} = n - 2\frac{n}{4}$ (tercera vuelta)
4. $j = n - 2\frac{n}{4} - \frac{n}{4} = n - 3\frac{n}{4}$ (cuarta vuelta)
5. $j = n - 3\frac{n}{4} - \frac{n}{4} = n - 4\frac{n}{4} = n - n = 0$ (Esta vez ya no se ejecuta)

El for interno se ejecuta 4 veces, una cantidad constante. La sumatoria nos queda

$$\sum_{j=1}^4 c_2$$

- Sumatoria externa

Este for deja de ejecutarse cuando $i = n^2$ y aumenta en $i + 2$ en cada iteración. Si probamos con distintos valores de n .

– Con $n = 3$

1. $i = 0$
2. $i = 2$
3. $i = 6$
4. $i = 8$

– Con $n = 4$

1. $i = 0$
2. $i = 2$
3. $i = 4$
4. $i = 8$
5. $i = 10$
6. $i = 12$
7. $i = 14$
8. $i = 16$

El for externo se ejecuta $\frac{n^2}{2}$ veces.

$$\sum_{i=0}^{\frac{n^2}{2}} \left(\sum_{j=1}^4 c_2 \right)$$

Podemos armar una expresión de $T(n)$.

$$T(n) = c_1 + \sum_{i=0}^{\frac{n^2}{2}} \left(\sum_{j=1}^4 c_2 \right)$$

Ahora, tenemos que "deshacernos" de las sumatorias y hacer que todo quede en función de n . Propiedades a usar:

- $\sum_{k=m}^n c = c(n - m + 1)$

Entonces,

$$\begin{aligned} T(n) &= c_1 + \sum_{i=0}^{\frac{n^2}{2}} \left(\sum_{j=1}^4 c_2 \right) \\ &= c_1 + \sum_{i=0}^{\frac{n^2}{2}} 4c_2 = \\ &= c_1 + 4c_2 \left(\frac{n^2}{2} - 0 + 1 \right) = \\ &= c_1 + 4c_2 + 2c_2n^2 \end{aligned}$$

Si renombramos:

- $c_3 = c_1 + 4c_2$
- $c_4 = 2c_2$

Tenemos el tiempo de ejecución T en función de n .

$$T(n) = c_3 + c_4 n^2$$

b. Para establecer el orden de la función, hay que hacer la demostración. Podemos decir que n^2 crece más rápido que n . Se tiene que demostrar que

$$T(n) = c_3 + c_4 n^2 \leq O(n^2)$$

Para esto, tenemos que encontrar una constante $k \geq 0$ y un n_0 tales que

$$c_3 + c_4 n^2 \leq k n^2 \text{ para todo } n \geq n_0$$

Empezamos analizando término a término.

- Primer término: c_3
Sabemos que $1 \leq n^2$. Si multiplicamos a cada lado por c_3 .

$$c_3 \leq c_3 n^2$$

Encontramos entonces un valor k_1 que acota al término, $k_1 = c_3$. Si probamos con $n_0 = 0$, la desigualdad no se cumple porque $c_3 \geq c_3(0)^2$. Para $n_0 = 1$,

$$c_3 \leq c_3 \cdot 1^2 \leq c_3$$

La desigualdad se cumple. El término se puede acotar con $k_1 = c_3$ y $n_0 = 1$, para $n_0 \geq 1$.

- Segundo término: $c_4 n^2$
Sabemos que $n^2 \leq n^2$. Si multiplicamos a ambos lados por c_4 ,

$$c_4 n^2 \leq c_4 n^2$$

Encontramos un valor para $k_2 = c_4$. Si tomamos $n_0 = 0$,

$$c_4 \cdot 0^2 \leq c_4 \cdot 0^2$$

La desigualdad se cumple. El término se puede acotar con $k_2 = c_4$ y $n_0 = 0$, para $n_0 \geq 0$.

Ahora, sumamos todos los resultados parciales que obtuvimos antes.

$$c_3 + c_4 n^2 \leq k_1 n^2 + k_2 n^2$$

$$T(n) \leq (k_1 + k_2) \cdot n^2$$

$$T(n) \leq (c_3 + c_4) \cdot n^2$$

Renombrando $c = c_3 + c_4$

$$T(n) \leq c n^2$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(n^2), \text{ con } c = c_3 + c_4 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$

3)

```
public static void calcular(int n) {
    int i, j, r = 0;
    for ( i = 1; i < n; i= i+2)
        for (j = 1; j <= i; j++ )
            r = r + 1;

    return r;
}
```

a. En este algoritmo tenemos, en la primera línea, operaciones constante que vamos a representar con c_1 y dos for anidados que van a ser traducidas como una sumatoria con otra adentro.

- La sumatoria interna (el segundo for), se va ejecutar i veces (teniendo en cuenta también, que el índice aumenta de uno en uno), realizándose una operación constante. Vamos a tomar los límites de la sumatoria desde $j = 1$ hasta i .

$$\sum_{j=1}^i c_2$$

- La sumatoria externa se ejecuta desde n hasta el valor anterior a n . El índice cambia sumándose en 2 luego de cada iteración. Veamos como cambia el índice en cada iteración:

1. $i = 1$
2. $i = 1 + 2$
3. $i = 1 + 2 + 2 = 1 + 2 \cdot 2$
4. $i = 1 + 2 \cdot 2 + 2 = 1 + 2 \cdot 3$
- $\vdots$

$k.$ $i = 1 + 2 \cdot (k - 1)$ Paso general

Como el for se ejecuta hasta que $i = n$, despejamos n del paso general para saber cuándo el índice llega al valor de $n - 1$. Agarré $n - 1$ porque nunca la condición de corte es $i < n$, no

$$i \leq n.$$

$$n - 1 = 1 + 2(k - 1)$$

$$n - 1 = 2k - 2$$

$$\frac{n+1}{2} = k \quad \text{Este es el límite superior de la sumatoria}$$

Entonces, la sumatoria del primer for queda,

$$\sum_{i=1}^{\frac{n+1}{2}} \left(\sum_{j=1}^i c_2 \right)$$

Con esto, se puede armar una expresión de $T(n)$.

$$T(n) = c_1 + \sum_{i=1}^{\frac{n+1}{2}} \left(\sum_{j=1}^i c_2 \right)$$

Ahora, tenemos que "deshacernos" de las sumatorias y hacer que todo quede en función de n . Propiedades a usar:

- $\sum_{i=1}^n c = (n - 1 + 1) \times c = n \times c$
- $\sum_{i=1}^n c \cdot a_i = c \times \sum_{i=1}^n a_i$
- $\sum_{i=1}^n i = \frac{n(n+1)}{2}$

Entonces,

$$\begin{aligned} T(n) &= c_1 + \sum_{i=1}^{\frac{n+1}{2}} \left(\sum_{j=1}^i c_2 \right) \\ &= c_1 + \sum_{i=1}^{\frac{n+1}{2}} (c_2 \cdot (i - 1 + 1)) \\ &= c_1 + \sum_{i=1}^{\frac{n+1}{2}} c_2 i \\ &= c_1 + c_2 \sum_{i=1}^{\frac{n+1}{2}} i \\ &= c_1 + c_2 \cdot \left(\frac{\frac{n+1}{2} \cdot (\frac{n+1}{2} + 1)}{2} \right) \\ &= c_1 + c_2 \cdot \left(\left(\frac{n+1}{2} \right) \left(\frac{n+3}{4} \right) \right) \\ &= c_1 + c_2 \frac{1}{8} \cdot (n^2 + 4n + 3) \\ &= c_1 + \frac{3}{8}c_2 + \frac{1}{2}c_2 n + \frac{1}{8}c_2 n^2 \end{aligned}$$

Renombrando

- $c_3 = c_1 + \frac{3}{8}c_2$
- $c_4 = \frac{1}{2}c_2$
- $c_5 = \frac{1}{8}c_2$

Tenemos el tiempo de ejecución T en función de n .

$$T(n) = c_3 + c_4n + c_5n^2$$

b. Para establecer el orden de la función, hay que hacer la demostración. Podemos decir que n^2 crece más rápido que n . Se tiene que demostrar que

$$T(n) = c_3 + c_4n + c_5n^2 \leq O(n^2)$$

Para esto, tenemos que encontrar una constante $k \geq 0$ y un n_0 tales que

$$c_3 + c_4n + c_5n^2 \leq kn^2 \text{ para todo } n \geq n_0$$

Empezamos analizando término a término.

- Primer término: c_3
Sabemos que $1 \leq n^2$. Si multiplicamos a cada lado por c_3 .

$$c_3 \leq c_3n^2$$

Encontramos entonces un valor k_1 que acota al término, $k_1 = c_3$. Si probamos con $n_0 = 0$, la desigualdad no se cumple porque $c_3 \geq c_3(0)^2$. Para $n_0 = 1$,

$$c_3 \leq c_3 \cdot 1^2 \leq c_3$$

La desigualdad se cumple. El término se puede acotar con $k_1 = c_3$ y $n_0 = 1$, para $n_0 \geq 1$.

- Segundo término: c_4n
Sabemos que $n \leq n^2$. Si multiplicamos a ambos lados por c_4 ,

$$c_4n \leq c_4n^2$$

Encontramos un valor para $k_2 = c_4$. Si tomamos $n_0 = 0$,

$$c_4 \cdot 0 \leq c_4 \cdot 0^2$$

La desigualdad se cumple. El término se puede acotar con $k_2 = c_4$ y $n_0 = 0$, para $n_0 \geq 0$.

- Tercer término: $c_5 n^2$
Sabemos que $n^2 \leq n^2$. Si multiplicamos a ambos lados por c_5 ,

$$c_5 n^2 \leq c_5 n^2$$

Encontramos un valor para $k_3 = c_5$. Si tomamos $n_0 = 0$,

$$c_5 \cdot 0^2 \leq c_5 \cdot 0^2$$

La desigualdad se cumple. El término se puede acotar con $k_3 = c_5$ y $n_0 = 0$, para $n_0 \geq 0$.

Ahora, sumamos todos los resultados parciales que obtuvimos antes.

$$c_3 + c_4 n + c_5 n^2 \leq k_1 n^2 + k_2 n^2 + k_3 n^2$$

$$T(n) \leq (k_1 + k_2 + k_3) \cdot n^2$$

$$T(n) \leq (c_3 + c_4 + c_5) \cdot n^2$$

Renombrando $c = c_3 + c_4 + c_5$

$$T(n) \leq c n^2$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(n^2), \text{ con } c = c_3 + c_4 + c_5 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$

8 Ejercicio 8

- ¿Qué hace cada uno de los métodos que aparece a continuación?
- Calcule la función de tiempo de ejecución y el orden de la misma para cada método.
- ¿Cuál de los dos métodos es más eficiente? ¿Por qué?

```
static public int pIter(int x, int n){
    int potencia;
    if (n == 0)
        p = 1;
    else {
        if (n == 1)
            p = x;
        else{
            p = x;
            for (int i = 2 ; i <= n ; i++) {
                p *= x ;
            }
        }
    }
}
```

```

    }
}
return p;
}

static public int pRec( int x, int n){
    if( n == 0 )
        return 1;
    else{
        if( n == 1)
            return x;
        else{
            if ( (n % 2 ) == 0)
                return pRec (x * x, n / 2 );
            else
                return pRec (x * x, n / 2) * x;
        }
    }
}
}

```

- a. Los métodos calculan la potencia de orden n de x . La diferencia es que `pIter` lo hace iterativamente mientras que `pRec` lo hace recursivamente.
- b. Calculemos las funciones y el orden de ejecución de ambos métodos.

- pIter

Primero tenemos varias asignaciones que son operaciones constantes y las vamos a representar con la constante c_1 . Después tenemos un `for` que vamos a representar mediante una sumatoria.

Como el primer valor que tomar i es 2, ese va a ser el límite inferior de la sumatoria. Además, el índice aumenta de a uno, así que podemos tomar el límite superior como n .

La sumatoria nos va a quedar,

$$\sum_{i=2}^n c_2$$

Podemos escribir la función $T(n)$ como

$$T(n) = c_1 + \sum_{i=2}^n c_2$$

Ahora vamos a usar trucos matemáticos para hacer volar las variables de iteración. Propiedades a usar:

$$- \sum_{k=m}^n c = c \cdot (n - m + 1)$$

Entonces,

$$\begin{aligned} T(n) &= c_1 + \sum_{i=2}^n c_2 \\ &= c_1 + c_2(n - 2 + 1) \\ &= c_1 - c_2 + c_2n \end{aligned}$$

$$\text{Renombrando } c_3 = c_1 - c_2.$$

$$= c_3 + c_2n$$

Función de tiempo de ejecución:

$$\boxed{T(n) = c_3 + c_2n}$$

Vamos a demostrar que

$$\boxed{T(n) = c_3 + c_2n \leq O(n)}$$

buscando los valores de k y n_0 que acoten la función.

- Primer término c_3 Sabemos que $1 \leq n$. Si multiplicamos a ambos lados por c_3 .

$$c_3 \leq c_3 \cdot n$$

Ya tenemos el valor de $k_1 = c_3$. Si tomamos $n_0 = 1$, la desigualdad se cumple

$$c_3 \leq c_3(1)$$

- Segundo término Sabemos que $n \leq n$. Si multiplicamos a ambos lados por c_2 .

$$c_2 \cdot n \leq c_2 \cdot n$$

Ya tenemos el valor de $k_2 = c_2$. Si tomamos $n_0 = 0$, la desigualdad se cumple

$$c_2 \cdot 0 \leq c_2 \cdot 0$$

Sumamos todos los resultados parciales.

$$c_3 + c_2n \leq k_1n + k_2n$$

$$T(n) \leq (k_1 + k_2)n$$

$$T(n) \leq (c_3 + c_2)n$$

$$\text{Renombrando } c = c_3 + c_2$$

$$T(n) \leq cn$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(n), \text{ con } c = c_3 + c_2 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$

- pRecur Tenemos un método recursivo con dos casos base que realizan una función constante.

El caso base lo representamos con c_1 para los $n = 0$ y $n = 1$.

$$c_1 \quad n \leq 1$$

Por otro lado, para otros valores de n , se realiza una llamada recursiva.

$$c_2 + T\left(\frac{n}{2}\right) \quad n > 1$$

Podemos escribir la función $T(n)$ como

$$T(n) = \begin{cases} c_1 & n \leq 1, \\ c_2 + T\left(\frac{n}{2}\right) & n > 1. \end{cases}$$

Ahora hay que ver como va cambiando la expresión con las llamadas recursivas para llegar a un paso general.

1. $T(n) = c_2 + T\left(\frac{n}{2}\right)$
 2. $T(n) = c_2 + (c_2 + T\left(\frac{n}{2 \cdot 2}\right)) = 2c_2 + T\left(\frac{n}{2^2}\right)$
 3. $T(n) = 2c_2 + (c_2 + T\left(\frac{n}{2 \cdot 2^2}\right)) = 3c_2 + T\left(\frac{n}{2^3}\right)$
- $\vdots$

Paso i (paso general) $T(n) = i \cdot c_2 + T\left(\frac{n}{2^i}\right)$

Ahora hay que despejar i de la expresión que quedó "como parámetro" a la llamada recursiva para obtener el caso base.

Como el caso base se ejecuta cuando $n \leq 1$, vamos a tomar el 1.

O sea,

$$\begin{aligned} \frac{n}{2^i} &= 1 \\ n &= 2^i \\ i &= \log_2 n \end{aligned}$$

Reemplazamos el caso base en la función que obtuvimos como paso general. Propiedades usadas:

$$- a^{\log_a b} = b$$

$$T(n) = \log_2 n \cdot c_2 + T\left(\frac{n}{2^{\log_2 n}}\right)$$

$$T(n) = c_2 \cdot \log_2 n + T(1)$$

$$T(n) = c_2 \cdot \log_2 n + c_1$$

Entonces, la función de tiempo de ejecución de pRecur es

$$\boxed{T(n) = c_2 \cdot \log_2 n + c_1}$$

Vamos a demostrar el orden de la función. Hay que demostrar que

$$T(n) = c_2 \cdot \log_2 n + c_1 \leq O(\log_2(n))$$

Hay que buscar los valores constantes c y n_0 tales que

$$c_2 \cdot \log_2 n + c_1 \leq c \cdot \log_2(n), \text{ para todo } n \geq n_0$$

– Primer término, $c_2 \cdot \log_2 n$

$$\log_2 n \leq \log_2 n$$

Multiplicamos ambos términos por c_2 .

$$c_2 \cdot \log_2 n \leq c_2 \cdot \log_2 n$$

Con esto hallamos el valor de k_1 , es $k_1 = c_2$. Esta desigualdad vale para $n_0 = 1$ (el logaritmo no está definido para $n_0 = 0$).

– Segundo término: c_1 Sabemos que $1 \leq \log_2 n$. Si multiplicamos a ambos lados por c_1 .

$$c_1 \leq c_1 \log_2 n$$

Tenemos el valor de la constante $k_2 = c_1$. Y la desigualdad se cumple para $n_0 \geq 1$.

Sumamos todos los miembros los resultados anteriores de cada lado.

$$T(n) = c_2 \cdot \log_2 n + c_1 \leq k_1 \log_2 n + k_2 \log_2 n$$

$$T(n) \leq (k_1 + k_2) \cdot \log_2 n$$

$$T(n) \leq (c_2 + c_1) \cdot \log_2 n$$

$$\text{Renombrando } c = c_2 + c_1$$

$$T(n) \leq c \cdot \log_2 n$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(\log_2 n), \text{ con } c = c_2 + c_1 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1$$

c. El método pIter es más eficiente que pRec porque $O(n) > O(\log_2 n)$.

9 Ejercicio 9

- Expresa la función del tiempo de ejecución de cada uno de los siguientes algoritmos, resuélvala y calcule el orden.

- b. Compare el tiempo de ejecución del método 'rec2' con el del método 'rec1'.
- c. Implemente un algoritmo más eficiente que el del método 'rec3'. (es decir, que el $T(n)$ sea menor).

```
static public int rec1(int n){
    if (n <= 1)
        return 1;
    else
        return (rec1(n-1) + rec1(n-1));
}

static public int rec2(int n){
    if (n <= 1)
        return 1;
    else
        return (2 * rec2(n-1));
}

static public int rec3(int n){
    if ( n == 0 )
        return 0;
    else {
        if ( n == 1 )
            return 1;
        else
            return (rec3(n-2) * rec3(n-2));
    }
}
```

a.

rec1

Por un lado tenemos una operación constante para los valores de $n \leq 1$ que vamos a representar con la constante c_1 .

Por otro lado, para el resto de valores de n , vemos que se realizan dos llamadas recursivas enviando como parámetro $n - 1$:

$$c_2 + 2 \cdot T(n - 1)$$

Podemos expresar a $T(n)$ como

$$T(n) = \begin{cases} c_1 & n \leq 1, \\ c_2 + 2 \cdot T(n - 1) & n > 1. \end{cases}$$

Analizamos el desarrollo de la llamada recursiva para encontrar el paso general.

1. $T(n) = c_2 + 2 \cdot T(n-1)$
2. $T(n) = c_2 + 2(c_2 + 2 \cdot T(n-2)) = 3c_2 + 2 \cdot 2 \cdot T(n-2)$
3. $T(n) = 3c_2 + 2^2(c_2 + 2 \cdot T(n-3)) = 7c_2 + 2^2 \cdot 2 \cdot T(n-3)$
- $\vdots$

Paso i :

$$T(n) = (2^i - 1) \cdot c_2 + 2^i \cdot T(n-i)$$

Ahora hay que despejar i de la expresión que quedó "como parámetro" a la llamada recursiva para obtener el caso base.

Como el caso base se ejecuta cuando $n \leq 1$, vamos a tomar el 1.

O sea,

$$\begin{aligned} n - i &= 1 \\ -i &= 1 - n \\ i &= n - 1 \end{aligned}$$

Reemplazamos el caso base en la función que obtuvimos como paso general.

$$\begin{aligned} T(n) &= (2^{n-1} - 1) \cdot c_2 + 2^{n-1} \cdot T(n - n + 1) \\ &= 2^{n-1} \cdot c_2 - c_2 + 2^{n-1} \cdot T(1) \\ &= 2^n \cdot 2^{-1} \cdot c_2 - 2^n \cdot 2^{-1} \cdot c_1 - c_2 \\ &= 2^n \cdot (2^{-1}c_2 + 2^{-1}c_1) - c_2 \end{aligned}$$

Renombramos

- $c_3 = 2^{-1}c_2 + 2^{-1}c_1$
- $c_4 = -c_2$

$$= 2^n \cdot c_3 + c_4$$

Entonces, la función de tiempo de ejecución de `rec1` es

$$\boxed{T(n) = 2^n \cdot c_3 + c_4}$$

Vamos a demostrar el orden de la función. Hay que demostrar que

$$\boxed{T(n) = 2^n \cdot c_3 + c_4 \leq O(2^n)}$$

Hay que buscar los valores constantes c y n_0 tales que

$$2^n \cdot c_3 + c_4 \leq c \cdot 2^n, \text{ para todo } n \geq n_0$$

- Primer término, $2^n \cdot c_3$
Se cumple que $2^n \leq 2^n$. Si multiplicamos a ambos lados por c_3 .

$$2^n c_3 \leq 2^n c_3$$

Con esto hallamos el valor de k_1 , es $k_1 = c_3$. Esta desigualdad vale para $n_0 = 0$.

- Segundo término, c_4 Se cumple que $1 \leq 2^n$.
Si multiplicamos a ambos lados por c_4 .

$$c_4 \leq c_4 \cdot 2^n$$

Con esto hallamos el valor de k_2 , es $k_2 = c_4$. Con $n_0 = 0$ la desigualdad no se cumple, así que vamos a tomar $n_0 = 1$. La desigualdad se cumple para los $n \geq 1$.

Sumamos todos los miembros los resultados anteriores de cada lado.

$$2^n c_3 + c_2 \leq 2^n k_1 + 2^n k_2$$

$$T(n) \leq 2^n (k_1 + k_2)$$

$$T(n) \leq 2^n (c_3 + c_4)$$

$$\text{Renombramos, } c = c_3 + c_4$$

$$T(n) \leq 2^n \cdot c$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(2^n), \text{ con } c = c_3 + c_4 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1$$

rec2

Por un lado tenemos una operación constante para los valores de $n \leq 1$ que vamos a representar con la constante c_1 .

Por otro lado, para el resto de valores de n , vemos que se realiza una llamada recursiva enviando como parámetro $n - 1$:

$$c_2 + T(n - 1)$$

Podemos expresar a $T(n)$ como

$$T(n) = \begin{cases} c_1 & n \leq 1, \\ c_2 + T(n - 1) & n > 1. \end{cases}$$

Analizamos el desarrollo de la llamada recursiva para encontrar el paso general.

$$1. T(n) = c_2 + T(n - 1)$$

$$2. T(n) = c_2 + (c_2 + T(n-2)) = 2 \cdot c_2 + T(n-2)$$

$$3. T(n) = 2c_2 + (c_2 + T(n-3)) = 3c_2 + T(n-3)$$

$\vdots$

Paso i :

$$T(n) = i \cdot c_2 + T(n-i)$$

Ahora hay que despejar i de la expresión que quedó "como parámetro" a la llamada recursiva para obtener el caso base.

Como el caso base se ejecuta cuando $n \leq 1$, vamos a tomar el 1.

O sea,

$$n - i = 1$$

$$-i = 1 - n$$

$$i = n - 1$$

Reemplazamos el caso base en la función que obtuvimos como paso general.

$$T(n) = (n-1) \cdot c_2 + T(n-(n-1))$$

$$= c_2 \cdot n - c_2 + T(1)$$

$$= c_2 \cdot n - c_2 + c_1$$

$$\text{Renombrando } c_3 = -c_2 + c_1$$

$$= c_2 n + c_3$$

Entonces, la función de tiempo de ejecución de `rec1` es

$$\boxed{T(n) = c_2 n + c_3}$$

Vamos a demostrar el orden de la función. Hay que demostrar que

$$\boxed{T(n) = c_2 n + c_3 \leq O(n)}$$

Hay que buscar los valores constantes c y n_0 tales que

$$c_2 n + c_3 \leq c \cdot n, \text{ para todo } n \geq n_0$$

Primer término, $n \cdot c_2$

Se cumple que $n \leq n$. Si multiplicamos a ambos lados por c_2 .

$$nc_2 \leq nc_2$$

Con esto hallamos el valor de k_1 , es $k_1 = c_2$. Esta desigualdad vale para $n_0 = 0$.

Segundo término, c_3
 Se cumple que $1 \leq n$.
 Si multiplicamos a ambos lados por c_3 .

$$c_3 \leq c_3 \cdot n$$

Con esto hallamos el valor de k_2 , es $k_2 = c_3$. Con $n_0 = 0$ la desigualdad no se cumple, así que vamos a tomar $n_0 = 1$. La desigualdad se cumple para los $n \geq 1$.

Ahora sumamos a cada lado, los resultados parciales que obtuvimos.

$$c_2 \cdot n + c_3 \leq k_1 \cdot n + k_2 \cdot n$$

$$T(n) \leq (k_1 + k_2) \cdot n$$

$$T(n) \leq (c_2 + c_3) \cdot n$$

Renombrando $c = c_2 + c_3$

$$T(n) \leq c \cdot n$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(n), \text{ con } c = c_2 + c_3 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1$$

rec3

Por un lado tenemos una operación constante para los valores de $n \leq 1$ que vamos a representar con la constante c_1 .

Por otro lado, para el resto de valores de n , vemos que se realizan dos llamadas recursivas enviando como parámetro $n - 2$:

$$c_2 + 2 \cdot T(n - 2)$$

Podemos expresar a $T(n)$ como

$$T(n) = \begin{cases} c_1 & n \leq 1, \\ c_2 + 2 \cdot T(n - 2) & n > 1. \end{cases}$$

Analizamos el desarrollo de la llamada recursiva para encontrar el paso general.

1. $T(n) = c_2 + 2 \cdot T(n - 2)$
2. $T(n) = c_2 + 2(c_2 + 2 \cdot T(n - 4)) = 3c_2 + 2 \cdot 2 \cdot T(n - 4)$
3. $T(n) = 3c_2 + 2^2(c_2 + 2 \cdot T(n - 6)) = 7c_2 + 2^2 \cdot 2 \cdot T(n - 6)$
- $\vdots$

Paso i :

$$T(n) = (2^i - 1) \cdot c_2 + 2^i \cdot T(n - 2 \cdot i)$$

Ahora hay que despejar i de la expresión que quedó "como parámetro" a la llamada recursiva para obtener el caso base.
 Como el caso base se ejecuta cuando $n \leq 1$, vamos a tomar el 1.
 O sea,

$$\begin{aligned}n - 2 \cdot i &= 1 \\-2 \cdot i &= 1 - n \\i &= \frac{n - 1}{2}\end{aligned}$$

Reemplazamos el caso base en la función que obtuvimos como paso general.

$$\begin{aligned}T(n) &= (2^{\frac{n-1}{2}} - 1) \cdot c_2 + 2^{\frac{n-1}{2}} \cdot T(n - 2(\frac{n-1}{2})) \\&= (2^{\frac{n}{2}} \cdot 2^{-\frac{1}{2}} - 1) \cdot c_2 + 2^{\frac{n}{2}} \cdot 2^{-\frac{1}{2}} \cdot T(n - (n - 1)) \\&= 2^{\frac{n}{2}} \cdot 2^{-\frac{1}{2}} \cdot c_2 - c_2 + 2^{\frac{n}{2}} \cdot 2^{-\frac{1}{2}} \cdot c_1 \\&= 2^{\frac{n}{2}} \cdot (2^{-\frac{1}{2}} c_2 + 2^{-\frac{1}{2}} c_1) - c_2\end{aligned}$$

Renombrando

- $c_3 = 2^{-\frac{1}{2}} c_2 + 2^{-\frac{1}{2}} c_1$
- $c_4 = -c_2$

Entonces, la función de tiempo de ejecución de rec3 es

$$\boxed{T(n) = 2^{\frac{n}{2}} \cdot c_3 + c_4}$$

Otra forma de escribir la función es $T(n) = \sqrt{2^n} \cdot c_3 + c_4$. Vamos a demostrar el orden de la función. Hay que demostrar que

$$\boxed{T(n) = \sqrt{2^n} \cdot c_3 + c_4 \leq O(n)}$$

Hay que buscar los valores constantes c y n_0 tales que

$$\sqrt{2^n} \cdot c_3 + c_4 \leq \sqrt{2^n}, \text{ para todo } n \geq n_0$$

Analizamos término a término.

- Primer término, $\sqrt{2^n} \cdot c_3$
 Se cumple que $\sqrt{2^n}$. Si multiplicamos a ambos lados por c_3 .

$$\sqrt{2^n} \cdot c_3 \leq \sqrt{2^n} \cdot c_3$$

Con esto hallamos el valor de k_1 , es $k_1 = c_3$. Esta desigualdad vale para $n_0 = 0$. La desigualdad se cumple para los $n \geq 0$.

- Segundo término, c_4
Se cumple que, $1 \leq \sqrt{2^n}$. Si multiplicamos a ambos lados por c_4 .

$$c_4 \leq \sqrt{2^n} \cdot c_4$$

Con esto hallamos el valor de k_2 , es $k_2 = 1$. Esta desigualdad vale para $n_0 = 1$. La desigualdad se cumple para los $n \geq 1$.

Sumamos todos los miembros los resultados anteriores de cada lado.

$$\sqrt{2^n} \cdot c_3 + c_4 \leq \sqrt{2^n} \cdot k_1 + \sqrt{2^n} \cdot k_2$$

$$T(n) \leq (k_1 + k_2) \cdot \sqrt{2^n}$$

$$T(n) \leq (c_3 + c_4) \cdot \sqrt{2^n}$$

Renombrando $c = c_3 + c_4$

$$T(n) \leq c \cdot \sqrt{2^n}$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(\sqrt{2^n}), \text{ con } c = c_3 + c_4 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1$$

b. El método `rec2` es más eficiente que `rec1` porque $O(n) < O(2^n)$. Mientras más rápido crezca la función, menos eficiente es el algoritmo.

c.

```
static public int (){                                rec3_mejorado(int n){
    return n % 2;
}
```

Este método es más eficiente porque su orden de ejecución es constante, $O(1)$.

Los ejercicios 10 y 11 te las debo, no tenía ganas de hacerlos.

10 Ejercicio 12

Resuelva la función de tiempo de ejecución de los siguientes segmentos de código y calcule el orden.

<pre>int recur (int n){ if (n == 1) return 1; else return(recur(n/2) * recur(n/2)); }</pre>	<pre>int recursivo(int n){ if (n <= 5) return 1; else return (recursivo (n-5)); }</pre>
<pre>int recursivo(int n){ if (n <= 7) return 1; else return (recursivo (n/8)); }</pre>	

recur

Por un lado tenemos una operación constante para los valores de $n \leq 1$ que vamos a representar con la constante c_1 .

Por otro lado, para el resto de valores de n , vemos que se realizan dos llamadas recursivas enviando como parámetro $\frac{n}{2}$:

$$c_2 + 2 \cdot T\left(\frac{n}{2}\right)$$

Podemos expresar a $T(n)$ como

$$T(n) = \begin{cases} c_1 & n = 1, \\ c_2 + 2 \cdot T\left(\frac{n}{2}\right) & n > 1. \end{cases}$$

Analizamos el desarrollo de la llamada recursiva para encontrar el paso general.

1. $T(n) = c_2 + 2 \cdot T\left(\frac{n}{2}\right)$
2. $T(n) = c_2 + 2[c_2 + 2T\left(\frac{n}{2 \cdot 2}\right)] = 3c_2 + 2 \cdot 2T\left(\frac{n}{2}\right) = 3c_2 + 2^2T\left(\frac{n}{2^2}\right)$
3. $3c_2 + 2^2[c_2 + 2T\left(\frac{n}{2^2 \cdot 2}\right)] = 7 \cdot c_2 + 2^3 \cdot T\left(\frac{n}{2^3}\right)$

Paso i :

$$T(n) = (2^i - 1) \cdot c_2 + 2^i \cdot T\left(\frac{n}{2^i}\right)$$

Ahora hay que despejar i de la expresión que quedó "como parámetro" a la llamada recursiva para obtener el caso base.

Como el caso base se ejecuta cuando $n = 1$, vamos a tomar el 1.

O sea,

$$\begin{aligned} 1 &= \frac{n}{2^i} \\ 2^i &= n \\ i &= \log_2(n) \end{aligned}$$

Reemplazamos el caso base en la función que obtuvimos como paso general.

$$T(n) = (2^{\log_2 n} - 1) \cdot c_2 + 2^{\log_2 n} \cdot T\left(\frac{n}{2^{\log_2 n}}\right)$$

$$= (n - 1) \cdot c_2 + n \cdot T(1)$$

$$= c_2 \cdot n - c_2 + n \cdot c_1$$

$$\text{Renombramos, } c_3 = -c_2 \text{ y } c_4 = c_1 + c_2$$

$$= (c_1 + c_2) \cdot n + c_3$$

$$= c_4 \cdot n + c_3$$

Entonces, la función de tiempo de ejecución de rec1 es

$$\boxed{T(n) = c_4 \cdot n + c_3}$$

Vamos a demostrar el orden de la función. Hay que demostrar que

$$\boxed{T(n) = c_4 \cdot n + c_3 \leq O(n)}$$

Hay que buscar los valores constantes c y n_0 tales que

$$c_4 \cdot n + c_3 \leq c \cdot n, \text{ para todo } n \geq n_0$$

- Primer término, $c_4 \cdot n$

Se cumple que $n \leq n$. Si multiplicamos a ambos lados por c_4 .

$$c_4 \cdot n \leq c_4 \cdot n$$

Con esto hallamos el valor de k_1 , es $k_1 = c_4$. Esta desigualdad vale para $n_0 = 0$.

- Segundo término, c_3

Se cumple que $1 \leq n$.

Si multiplicamos a ambos lados por c_3 .

$$c_3 \leq c_3 n$$

Hallamos el valor de k_2 , $k_2 = c_3$. La desigualdad se cumple para los $n \geq 1$.
Tomamos $n_0 = 1$.

Ahora sumamos a cada lado, los resultados parciales que obtuvimos.

$$c_4 \cdot n + c_3 \leq k_1 \cdot n + k_2 \cdot n$$

$$T(n) \leq (k_1 + k_2) \cdot n$$

$$T(n) \leq (c_4 + c_3) \cdot n$$

$$\text{Renombrando } c = c_4 + c_3$$

$$T(n) \leq c \cdot n$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(n), \text{ con } c = c_4 + c_3 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1$$

recursivo

```
int recursivo(int n){
    if (n <= 5)
        return 1;
    else
        return (recursivo (n-5));
}
```

Por un lado tenemos una operación constante para los valores de $n \leq 5$ que vamos a representar con la constante c_1 .

Por otro lado, para el resto de valores de n , vemos que se realiza una llamada recursiva enviando como parámetro $n - 5$:

$$c_2 + T(n - 5)$$

Podemos expresar a $T(n)$ como

$$T(n) = \begin{cases} c_1 & n \leq 5, \\ c_2 + T(n - 5) & n > 5. \end{cases}$$

Analizamos el desarrollo de la llamada recursiva para encontrar el paso general.

1. $T(n) = c_2 + T(n - 5)$
2. $T(n) = c_2 + (c_2 + T(n - 10)) = 2 \cdot c_2 + T(n - 10)$
3. $T(n) = 2c_2 + (c_2 + T(n - 15)) = 3c_2 + T(n - 15)$
- $\vdots$

Paso i :

$$T(n) = i \cdot c_2 + T(n - 5i)$$

Ahora hay que despejar i de la expresión que quedó "como parámetro" a la llamada recursiva para obtener el caso base.

Como el caso base se ejecuta cuando $n \leq 5$, vamos a tomar el 5.

O sea,

$$\begin{aligned} n - 5i &= 5 \\ -5i &= 5 - n \\ i &= \frac{n - 5}{5} \end{aligned}$$

Reemplazamos el caso base en la función que obtuvimos como paso general.

$$\begin{aligned} T(n) &= \left(\frac{n-5}{5}\right) \cdot c_2 + T(n-5 \cdot \left(\frac{n-5}{5}\right)) \\ &= \frac{c_2}{5}n - c_2 + c_1 \end{aligned}$$

$$\begin{aligned} \text{Renombramos } c_3 &= c_1 - c_2 \text{ y } c_4 = \frac{c_2}{5} \\ &= c_4n + c_3 \end{aligned}$$

Entonces, la función de tiempo de ejecución de rec1 es

$$\boxed{T(n) = c_4 \cdot n + c_3}$$

Vamos a demostrar el orden de la función. Hay que demostrar que

$$\boxed{T(n) = c_4 \cdot n + c_3 \leq O(n)}$$

Hay que buscar los valores constantes c y n_0 tales que

$$c_4 \cdot n + c_3 \leq c \cdot n, \text{ para todo } n \geq n_0$$

- Primer término, $c_4 \cdot n$
Se cumple que $n \leq n$. Si multiplicamos a ambos lados por c_4 .

$$c_4 \cdot n \leq c_4 \cdot n$$

Con esto hallamos el valor de k_1 , es $k_1 = c_4$. Esta desigualdad vale para $n_0 = 0$.

- Segundo término, c_3
Se cumple que $1 \leq n$.
Si multiplicamos a ambos lados por c_3 .

$$c_3 \leq c_3n$$

Hallamos el valor de k_2 , $k_2 = c_3$. La desigualdad se cumple para los $n \geq 1$.
Tomamos $n_0 = 1$.

Ahora sumamos a cada lado, los resultados parciales que obtuvimos.

$$c_4 \cdot n + c_3 \leq k_1 \cdot n + k_2 \cdot n$$

$$T(n) \leq (k_1 + k_2) \cdot n$$

$$T(n) \leq (c_4 + c_3) \cdot n$$

$$\text{Renombrando } c = c_4 + c_3$$

$$T(n) \leq c \cdot n$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(n), \text{ con } c = c_4 + c_3 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1$$

recursivo


```

int recursivo(int n){
    if (n <= 7)
        return 1;
    else
        return (recursivo (n/8));
}

```

Por un lado tenemos una operación constante para los valores de $n \leq 7$ que vamos a representar con la constante c_1 .

Por otro lado, para el resto de valores de n , vemos que se realiza una llamada recursiva enviando como parámetro $\frac{n}{8}$:

$$c_2 + T\left(\frac{n}{8}\right)$$

Podemos expresar a $T(n)$ como

$$T(n) = \begin{cases} c_1 & n \leq 7, \\ c_2 + T\left(\frac{n}{8}\right) & n > 7. \end{cases}$$

Analizamos el desarrollo de la llamada recursiva para encontrar el paso general.

1. $T(n) = c_2 + T\left(\frac{n}{8}\right)$
2. $T(n) = c_2 + (c_2 + T\left(\frac{n}{8^2}\right)) = 2c_2 + T\left(\frac{n}{8^2}\right)$
3. $T(n) = 2c_2 + (c_2 + T\left(\frac{n}{8^3}\right)) = 3c_2 + T\left(\frac{n}{8^3}\right)$

Paso i , $T(n) = ic_2 + T\left(\frac{n}{8^i}\right)$

Ahora hay que despejar i de la expresión que quedó "como parámetro" a la llamada recursiva para obtener el caso base.

Como el caso base se ejecuta cuando $n \leq 7$, vamos a tomar el 7.

O sea,

$$\begin{aligned}
 7 &= \frac{n}{8^i} \\
 8^i &= \frac{n}{7} \\
 i &= \log_8\left(\frac{n}{7}\right)
 \end{aligned}$$

Reemplazamos el caso base en la función que obtuvimos como paso general.

Propiedades:

- $a^{\log_a(b)} = b$
- $\log_a\left(\frac{b}{c}\right) = \log_a b - \log_a c$

$$\begin{aligned}
T(n) &= \log_8\left(\frac{n}{7}\right)c_2 + T\left(\frac{n}{8^{\log_8(\frac{n}{7})}}\right) \\
&= \log_8\left(\frac{n}{7}\right)c_2 + c_1 \\
&= \log_8(n)c_2 - \log_8(7)c_2 + c_1 \\
\text{Renombrando } c_3 &= c_1 - \log_8(7)c_2 \\
&= \log_8(n)c_2 + c_3
\end{aligned}$$

Entonces, la función de tiempo de ejecución de rec1 es

$$\boxed{T(n) = \log_8(n)c_2 + c_3}$$

Vamos a demostrar el orden de la función. Hay que demostrar que

$$\boxed{T(n) = \log_8(n)c_2 + c_3 \leq O(\log_8(n))}$$

Hay que buscar los valores constantes c y n_0 tales que

$$\log_8(n)c_2 + c_3, \text{ para todo } n \geq n_0$$

Analizamos término a término

- Primer término, $\log_8 n \cdot c_2$
Se cumple que $\log_8 n \leq \log_8 n$.
Si multiplicamos a ambos lados por c_2 .

$$c_2 \log_8 n \leq c_2 \log_8 n$$

Con esto hallamos el valor de $k_1 = c_2$. Esta desigualdad vale para $n_0 = 1$.
El logaritmo está definido para $n \geq 1$.

- Segundo término, c_3
Se cumple que $1 \leq c_3$. Si multiplicamos a ambos lados por c_3 .

$$c_3 \leq c_3 \log_8 n$$

Con esto hallamos el valor de $k_2 = c_3$. Esta desigualdad vale para $n_0 = 1$.
El logaritmo no está definido para $n \geq 1$.

Ahora sumamos a cada lado, los resultados parciales que obtuvimos.

$$\log_8(n)c_2 + c_3 \leq k_1 \log_8 n + k_2 \log_8 n$$

$$T(n) \leq (k_1 + k_2) \log_8 n$$

$$T(n) \leq (c_2 + c_3) \log_8 n$$

$$\text{Renombramos } c = c_2 + c_3$$

$$T(n) \leq c \cdot \log_8 n$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(\log_8 n), \text{ con } c = c_2 + c_3 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1$$

11 Ejercicio 13

Considere el siguiente fragmento de código:

```
int count = 0; int n = a.length;
for (int i = 1; i <= n; i = i*2) {
    for (int j = 0; j < n; j+=n/2) {
        a[j]++;
    }
}
```

Este algoritmo se ejecuta en una computadora que procesa 1.000 operaciones por segundo. Determine el tiempo **aproximado** (considerando el orden de ejecución del algoritmo) que requerirá el algoritmo para resolver un problema de tamaño $n=4096$. Datos que tenemos:

- En un segundo se realizan 1000 operaciones.
- Cantidad de operaciones de la que tenemos que calcular el tiempo que tarda en ejecutarse, $n = 4096$.

Primero hay que calcular el orden del fragmento de código.

En este algoritmo tenemos, en la primera línea, operaciones constante que vamos a representar con c_1 y dos for anidados que van a ser traducidas como una sumatoria con otra adentro.

- Analizamos el primer for
Se ejecuta desde 1 hasta n pero en cada iteración se multiplica por 2.
Cambio del índice

1. $i = 1$
2. $i = 1 \cdot 2 \cdot 2 = 2^2$
3. $i = 1 \cdot 2 \cdot 2 \cdot 2 = 2^3$
- $\vdots$

Paso general: $k. \quad i = 2^{(k-1)}$

Como la sumatoria aumenta de uno en uno, hay que encontrar el límite superior de la sumatoria que se ajuste al último valor que va a tomar el índice. Despejamos i .

$$\begin{aligned}n &= i \\2^{k-1} &= i \\k-1 &= \log_2(i) \\k &= 1 + \log_2(i)\end{aligned}$$

Entonces la primera sumatoria se ejecuta desde $i = 1$ hasta $1 + \log_2(i)$.
La sumatoria queda:

$$\sum_{i=1}^{1+\log_2 n} (algo)$$

- Analizamos el segundo for
Se ejecuta desde $j = 0$ hasta el valor anterior más cercanos a n que pueda tomar j pero en cada iteración se le suma $\frac{n}{2}$ al índice.
Cambio del índice:

1. $j = 0$
2. $j = \frac{n}{2}$
3. $j = \frac{n}{2} + \frac{n}{2} = 2 \cdot \frac{n}{2} = n$

El segundo for se ejecuta 2 veces. La sumatoria queda

$$\sum_{j=0}^1 c_2$$

Entonces la función $T(n)$ queda:

$$T(n) = c_1 + \sum_{i=1}^{1+\log_2 n} \left(\sum_{j=0}^1 c_2 \right)$$

Ahora hay que eliminar todas las variables de cambio de índices. Propiedad a utilizar:

- $\sum_{i=0}^n c = (n - 1 + 1) \times c$

$$\begin{aligned} T(n) &= c_1 + \sum_{i=1}^{1+\log_2 n} \left(\sum_{j=0}^1 c_2 \right) \\ &= c_1 + \sum_{i=1}^{1+\log_2 n} (2 \cdot c_2) \\ &= c_1 + 2c_2 \cdot (\log_2(n) + 1 - 1) \\ &= c_1 + 2c_2 + 2c_2 \log_2 n \end{aligned}$$

El orden de la función de tiempo de ejecución para este algoritmo es de $\log_2 n$. Como es el tiempo **aproximado**, lo utilizamos para calcular la cantidad de operaciones para ese n .

$$\log_2 4096 = 12 \text{ operaciones}$$

Sabemos que se procesan 1000 operaciones en 1 segundo.

1000 operaciones	_____	1 segundo
12 operaciones	_____	$x = 0.012$ segundos

Por lo tanto, el tiempo aproximado para resolver ese problema es de 0.012 segundos.

12 Ejercicio 14

Dado el siguiente algoritmo:

```
public static int recursivo( int n ) {  
    if (n <= 1){  
        return 1;  
    }  
    int i = 1;  
    while (i <= n) {  
        i*=2;  
    }  
    return n + recursivo(n/2) * n/2;  
}
```

Calcular la función de tiempo de ejecución y el orden de ejecución de la misma.

- Primero tenemos el caso base que es una operación constante y representamos con c_1 .
- Después tenemos una iteración que se ejecuta desde 1 hasta n , inclusive. En cada iteración, el índice aumenta multiplicándose por 2 en cada vuelta. Cambio del índice:

1. $i = 1$
2. $i = 1 \cdot 2$
3. $i = 1 \cdot 2 \cdot 2 = 2^2$
4. $i = 1 \cdot 2 \cdot 2 \cdot 2 = 2^3$

Término general: $k. \quad i = 2^{k-1}$

Como la sumatoria aumenta de uno en uno, hay que encontrar el límite superior de la sumatoria que se ajuste al último valor que va a tomar el índice.

Despejamos i .

$$\begin{aligned}n &= i \\2^{k-1} &= i \\k - 1 &= \log_2(i) \\k &= 1 + \log_2(i)\end{aligned}$$

Cada iteración se ejecuta $1 + \log_2 n$ veces. La sumatoria queda

$$\sum_{i=1}^{1+\log_2 n} (c_2)$$

- Por último, tenemos una llamada recursiva que envía como parámetro $\frac{n}{2}$. La sumatoria queda,

$$T(n) = \begin{cases} c_1 & n \leq 1 \\ c_2 + \sum_{i=1}^{1+\log_2 n} c_2 + T(\frac{n}{2}) & n > 1 \end{cases}$$

Para facilitar el desarrollo de la recursión, utilizando esta propiedad

$$\sum_{k=m}^n c = c \cdot (n - m + 1)$$

podemos reescribir el segundo término,

$$\sum_{i=1}^{1+\log_2 n} c_2 = c_3(1 + \log_2 n - 1 + 1) = c_3(1 + \log_2 n)$$

Desarrollo de la recursión

1. $T(n) = c_2 + c_3(1 + \log_2 n) + T(\frac{n}{2})$
2. $T(n) = c_2 + c_3(1 + \log_2 n) + (c_2 + c_3(1 + \log_2 n) + T(\frac{n}{2^2})) = 2c_2 + 2c_3(\log_2 n) + T(\frac{n}{2^2})$
3. $T(n) = 2c_2 + 2c_3(\log_2 n) + (c_2 + c_3(1 + \log_2 n) + T(\frac{n}{2^2 \cdot 2})) = 3c_2 + 3c_3(1 + \log_2 n) + T(\frac{n}{2^3})$

Paso i : $T(n) = i \cdot c_2 + i \cdot c_3(1 + \log_2 n) + T(\frac{n}{2^i})$

Ahora hay que despejar i de la expresión que quedó "como parámetro" a la llamada recursiva para obtener el caso base.

Como el caso base se ejecuta cuando $n \leq 1$, vamos a tomar el 1.

O sea,

$$\begin{aligned} 1 &= \frac{n}{2^i} \\ 2^i &= n \\ i &= \log_2 n \end{aligned}$$

Reemplazamos el caso base en la función que obtuvimos como paso general. Propiedad a usar:

- $a^{\log_a(b)} = b$

$$\begin{aligned}
T(n) &= c_2 \log_2 n + c_3 \log_2 n(1 + \log_2 n) + T\left(\frac{n}{2^{\log_2 n}}\right) \\
&= c_2 \log_2 n + c_3 \log_2 n + c_2 \log_2^2 n + T(1) \\
&= \log_2 n \cdot (c_2 + c_3) + c_3 \log_2^2 n + c_1
\end{aligned}$$

Renombramos $c_4 = c_2 + c_3$

$$= \log_2 n \cdot c_4 + c_3 \log_2^2 n + c_1$$

Entonces, la función de tiempo de ejecución del método es

$$T(n) = \log_2 n \cdot c_4 + c_3 \log_2^2 n + c_1$$

Vamos a demostrar el orden de la función. Hay que demostrar que

$$T(n) = \log_2 n \cdot c_4 + c_3 \log_2^2 n + c_1 \leq O(\log_2^2 n)$$

Hay que buscar los valores constantes c y n_0 tales que

$$\log_2 n \cdot c_4 + c_3 \log_2^2 n + c_1, \text{ para todo } n \geq n_0$$

- Primer término, $\log_2 n \cdot c_4$
Sabemos que $\log_2 n \leq \log_2^2 n$.
Si multiplicamos a ambos lados de la desigualdad por c_4 .

$$c_4 \log_2 n \leq c_4 \log_2^2 n$$

Encontramos un valor k_1 que acota al término. Si tomamos $n_0 = 1$.

$$\begin{aligned}
c_4 \log_2 1 &\leq c_4 \log_2^2 1 \\
c_4 \cdot 0 &\leq c_4 \cdot 0 \\
0 &\leq 0
\end{aligned}$$

Tenemos el $k_1 = c_4$ y $n_0 = 1$ que acotan el término.

- Segundo término, $c_3 \log_2^2 n$
Sabemos que $\log_2^2 n \leq \log_2^2 n$.
Si multiplicamos a ambos lados de la desigualdad por c_3 .

$$c_3 \log_2^2 n \leq c_3 \log_2^2 n$$

Encontramos un valor k_2 que acota al término. Si tomamos $n_0 = 1$.

$$\begin{aligned}
c_3 \log_2^2 1 &\leq c_3 \log_2^2 1 \\
c_3 \cdot 0 &\leq c_3 \cdot 0 \\
0 &\leq 0
\end{aligned}$$

Tenemos el $k_2 = c_3$ y $n_0 = 1$ que acotan el término.

- Tercer término, c_1
Sabemos que $1 \leq \log_2^2 n$.
Si multiplicamos a ambos lados de la desigualdad por c_1 .

$$c_1 \leq c_1 \log_2^2 n$$

Encontramos un valor k_3 que acota al término. Si tomamos $n_0 = 2$.

$$c_1 \log_2 2 \leq c_1 \log_2^2 2$$

$$c_1 \cdot 1 \leq c_1 \cdot 1$$

$$c_1 \leq c_1$$

Tenemos el $k_1 = c_1$ y $n_0 = 2$ que acotan el término.

Ahora sumamos a cada lado, los resultados parciales que obtuvimos.

$$\log_2 n \cdot c_4 + c_3 \log_2^2 n + c_1 \leq k_1 \log_2^2 n + k_2 \log_2^2 n + k_3 \log_2^2 n$$

$$T(n) \leq (k_1 + k_2 + k_3) \log_2^2 n$$

$$T(n) \leq (c_4 + c_3 + c_1) \log_2^2 n$$

$$\text{Renombramos } c = c_4 + c_3 + c_1$$

$$T(n) \leq c \cdot \log_2^2 n$$

El n_0 más restrictivo es $n_0 = 2$ porque cumple con todos los términos para $n_0 \geq 2$. Por lo tanto,

$$T(n) \leq O(\log_2^2 n), \text{ con } c = c_4 + c_3 + c_1 \text{ para todo } n \geq n_0, \text{ con } n_0 = 2$$

13 Ejercicio 15

Dado el siguiente fragmento de código:

```
int resta = algún valor;
for (int i = n * n; i <= n * n * n; i += n * n)
    for (int j = 1; j <= i; j++)
        resta = resta - j;
```

Calcular la función de tiempo de ejecución y el orden de ejecución de la misma. Tenemos el primer for, que va desde $n \cdot n$ hasta $n \cdot n \cdot n$. Cambio del índice.

1. $i = n \cdot n = n^2$
2. $i = n^2 + n \cdot n = 2n^2$
3. $2n^2 + n \cdot n = 3n^2$

Paso general: $i = kn^2$

Como la sumatoria aumenta de uno en uno, hay que encontrar el límite superior de la sumatoria que se ajuste al último valor que va a tomar el índice.

$$\begin{aligned}n^3 &= i \\n^3 &= kn^2 \\n &= k\end{aligned}$$

La primera iteración se ejecuta n veces.

$$\sum_{i=1}^n algo$$

Como la cantidad de veces que se ejecuta el segundo for depende del valor de i , en esa iteración, en cada vuelta, el for interno se ejecuta $i \cdot n^2$ veces.

El n^2 , es porque i siempre toma el valor n^2 en cada iteración. La segunda sumatoria queda,

$$\sum_{j=1}^{i \cdot n^2} c_2$$

Propiedades a usar:

- $\sum_{i=1}^n c = (n - 1 + 1) \times c = n \times c$
- $\sum_{i=1}^n c \cdot a_i = c \times \sum_{i=1}^n a_i$
- $\sum_{i=1}^n i = \frac{n(n+1)}{2}$

Podemos expresar la función como

$$\begin{aligned}
 T(n) &= c_1 + \sum_{i=1}^n \left(\sum_{j=1}^{i \cdot n^2} c_2 \right) \\
 &= c_1 + \sum_{i=1}^n [c_2(i \cdot n^2 - 1 + 1)] \\
 &= c_1 + \sum_{i=1}^n [c_2 n^2 \cdot i] \\
 &= c_1 + c_2 n^2 \sum_{i=1}^n i \\
 &= c_1 + c_2 n^2 \left(\frac{n(n+1)}{2} \right) \\
 &= c_1 + \frac{c_2}{2} n^2 (n^2 + n) \\
 \text{Renombramos } c_3 &= \frac{c_2}{2} \\
 &= c_1 + c_3 n^2 (n^2 + n) \\
 &= c_1 + c_3 n^4 + c_3 n^3
 \end{aligned}$$

Entonces, la función de tiempo de ejecución del método es

$$T(n) = c_1 + c_3 n^4 + c_3 n^3$$

Vamos a demostrar el orden de la función. Hay que demostrar que

$$T(n) = c_1 + c_3 n^4 + c_3 n^3 \leq O(n^4)$$

Hay que buscar los valores constantes c y n_0 tales que

$$c_1 + c_3 n^4 + c_3 n^3, \text{ para todo } n \geq n_0$$

Analizamos término a término.

- Primer término, c_1
Sabemos que $1 \leq n^4$. Si multiplicamos a ambos lados por c_1 .

$$c_1 \leq c_1 n^4$$

Tenemos un k_1 que acota el término.

Con $n_0 = 1$, se cumple la desigualdad.

$$c_1 \leq c_1 \cdot 1$$

La desigualdad se cumple para $n \geq 1$.

- Segundo término, c_3n^4
Sabemos que $n^4 \leq n^4$. Si multiplicamos a ambos lados por c_3 .

$$c_3n^4 \leq c_3n^4$$

Tenemos un k_2 que acota el término.
Con $n_0 = 0$, se cumple la desigualdad.

$$c_3 \cdot 0 \leq c_3 \cdot 0$$

La desigualdad se cumple para $n \geq 0$.

- Tercer término, c_3n^3
Sabemos que $n^3 \leq n^4$. Si multiplicamos a ambos lados por c_3 .

$$c_3n^3 \leq c_3n^4$$

Tenemos un k_3 que acota el término.
Con $n_0 = 0$, se cumple la desigualdad.

$$c_3 \cdot 0 \leq c_3 \cdot 0$$

La desigualdad se cumple para $n \geq 0$.

Ahora sumamos a cada lado, los resultados parciales que obtuvimos.

$$c_1 + c_3n^4 + c_3n^3 \leq c_1n^4 + c_3n^4 + c_3n^4$$

$$T(n) \leq (c_1 + 2c_3)n^4$$

$$\text{Renombramos } c = c_1 + 2c_3$$

$$T(n) \leq cn^4$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(n^4), \text{ con } c = c_1 + 2c_3 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1$$

14 Ejercicio 16

¿Cuál es el orden de ejecución del siguiente método? Calcule la función de tiempo de ejecución

```
void fun(int n, int arr[])
{ int i = 0, j = 0;
  for (; i < n; ++i)
    while (j < n && arr[i] < arr[j])
      j++;
}
```

En este caso, el for se ejecuta n veces por i va aumentando de a 1.
 En esta iteración anidada también porque el peor caso es cuando j aumenta hasta n . Esto pasa sólo en el primer caso, por lo que tomo dos sumatorias separadas siendo,

- $\sum_{j=0}^n c_3$, la ejecución del while en la primera iteración (cuando $i = 0$) y
- $\sum_{i=1}^n c_3$, las ejecuciones restantes del for, desde $i = 1$ hasta n .

Entonces,

$$\begin{aligned} T(n) &= c_1 + \sum_{j=0}^n c_2 + \sum_{i=1}^n c_3 \\ &= c_1 + c_2(n+1) + c_3n \\ &= c_1 + c_2 + n(c_2 + c_3) \\ \text{Renombramos } c_4 &= c_1 + c_2 \text{ y } c_5 = c_2 + c_3 \\ &= c_4 + c_5n \end{aligned}$$

Entonces, la función de tiempo de ejecución del método es

$$\boxed{T(n) = c_5 \cdot n + c_4}$$

Vamos a demostrar el orden de la función. Hay que demostrar que

$$\boxed{T(n) = c_5 \cdot n + c_4 \leq O(n)}$$

Hay que buscar los valores constantes c y n_0 tales que

$$c_5 \cdot n + c_4 \leq c \cdot n, \text{ para todo } n \geq n_0$$

- Primer término, $c_5 \cdot n$
 Se cumple que $n \leq n$. Si multiplicamos a ambos lados por c_5 .

$$c_5 \cdot n \leq c_5 \cdot n$$

Con esto hallamos el valor de k_1 , es $k_1 = c_5$. Esta desigualdad vale para $n_0 = 0$.

- Segundo término, c_4
 Se cumple que $1 \leq n$.
 Si multiplicamos a ambos lados por c_4 .

$$c_4 \leq c_4n$$

Hallamos el valor de k_2 , $k_2 = c_4$. La desigualdad se cumple para los $n \geq 1$.
 Tomamos $n_0 = 1$.

Ahora sumamos a cada lado, los resultados parciales que obtuvimos.

$$\begin{aligned}
 c_5 \cdot n + c_4 &\leq k_1 \cdot n + k_2 \cdot n \\
 T(n) &\leq (k_1 + k_2) \cdot n \\
 T(n) &\leq (c_4 + c_5) \cdot n \\
 \text{Renombrando } c &= c_4 + c_5 \\
 T(n) &\leq c \cdot n
 \end{aligned}$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(n), \text{ con } c = c_4 + c_5 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1$$

15 Ejercicio 17

¿Cuál es el orden de ejecución del siguiente fragmento de código? Calcule la función de tiempo de ejecución.

```

for (int i = 0; i < n*n ; i ++)
    for (int j = i % n; j >= 0; j--)
        System.out.println("j:"+j);

```

Como el primer for va desde 0 hasta $i < n^2$ y el índice aumenta de a uno, la iteración se ejecuta $n^2 - 1$ veces. Por otro lado, en la sumatoria interna, al ser el primer valor del índice el resto de la división $\frac{i}{n}$. Si, por ejemplo, ponemos valores a n :

- Con $n = 2$
 1. Primera iteración
 - (a) $j = 1\%2 = 1$
 - (b) $j = 0$

Se ejecuta 2 veces
 2. Segunda iteración
 - (a) $j = 2\%2 = 0$

Se ejecuta una vez
 3. Tercera iteración
 - (a) $j = 3\%2 = 1$
 - (b) $j = 0$

Se ejecuta 2 veces
 4. cuarta iteración
 - (a) $j = 4\%2 = 0$

Se ejecuta una vez

- Con $n = 3$

1. Primera iteración

(a) $j = 1\%3 = 1$

(b) $j = 0$

Se ejecuta 2 veces

2. Segunda iteración

(a) $j = 2\%3 = 2$

(b) $j = 4\%3 = 1$

(c) $j = 0$

Se ejecuta 3 veces

3. Tercera iteración

(a) $j = 3\%3 = 0$

Se ejecuta 1 vez

4. Cuarta iteración

(a) $j = 4\%3 = 1$

(b) $j = 0$

Se ejecuta 2 veces

5. Quinta iteración

(a) $j = 5\%3 = 2$

(b) $j = 1$

(c) $j = 0$

Se ejecuta 3 veces

Esta me re costó jajaja.

El segundo for se ejecuta $i\%n + 1$ veces (el +1 es porque incluye el 0).

Entonces,

$$\begin{aligned}
 T(n) &= \sum_{i=0}^{n^2-1} \left[\sum_{j=1}^{i\%n+1} c_1 \right] \\
 &= \sum_{i=0}^{n^2-1} [c_1(i\%n + 1 - 1 + 1)] \\
 &= c_1 \sum_{i=0}^{n^2-1} (i\%n + 1)
 \end{aligned}$$

Como $i\%n$ genera un múltiples veces un patrón de 0, 1, 2, ..., n-1 repetido n veces, $\sum_{i=0}^{n^2-1} (i\%n + 1) = n \sum_{k=0}^{n-1} (k + 1)$.

Por ejemplo, en el caso de $n = 3$, tenemos 2 ejecuciones (de 0 a 1) en la primera iteración, 3 en la segunda (de 0 a 2) y 1 en la tercera (resto=0). Entonces,

$$\begin{aligned}
&= c_1 n \sum_{k=0}^{n-1} (k+1) \\
&= c_1 n \left[\sum_{k=0}^{n-1} k + \sum_{k=0}^{n-1} 1 \right] \\
&= c_1 n \left[\sum_{k=0}^0 k + \sum_{k=1}^{n-1} k + n \right] \\
&= c_1 n^2 + c_1 n \left[\frac{(n-1)n}{2} \right] \\
&= c_1 n^2 + \frac{c_1}{2} n(n^2 - n) \\
\text{Renombrando } c_2 &= \frac{c_1}{2} \text{ y } c_3 = -c_1 \\
&= c_1 n^2 + c_2 n^3 + c_3
\end{aligned}$$

Entonces, la función de tiempo de ejecución del método es

$$T(n) = c_1 n^2 + c_2 n^3 + c_3$$

Vamos a demostrar el orden de la función. Hay que demostrar que

$$T(n) = c_1 n^2 + c_2 n^3 + c_3 \leq O(n^3)$$

Hay que buscar los valores constantes c y n_0 tales que

$$c_1 n^2 + c_2 n^3 + c_3 \leq c \cdot n, \text{ para todo } n \geq n_0$$

Se tiene que demostrar que

$$T(n) = c_2 n^3 + c_1 n^2 + c_3 \leq O(n^3)$$

Para esto, tenemos que encontrar una constante $k > 0$ y un n_0 tales que

$$c_2 n^3 + c_1 n^2 + c_3 \leq k n^3 \text{ para todo } n \geq n_0$$

Empezamos analizando término a término.

- Primer término: $c_3 2^3$
Sabemos que $n^3 \leq n^3$ Si multiplicamos a ambos lados por c_2 ,

$$c_2 n^3 \leq c_2 n^3$$

Encontramos un valor $k_1 = c_2$. Ahora, si probamos con $n_0 = 0$.

$$c_2 \cdot 0^3 \leq c_2 \cdot 0^3$$

La desigualdad se cumple. El término se puede acotar con $k_1 = c_2$ y $n_0 = 0$, para $n_0 \geq 0$.

- Segundo término: $c_1 n^2$
Sabemos que $n^2 \leq n^3$ Si multiplicamos a ambos lados por c_1 ,

$$c_1 n^2 \leq c_1 n^3$$

Encontramos un valor $k_2 = c_1$. Ahora, si probamos con $n_0 = 0$.

$$c_1 \cdot 0^2 \leq c_1 \cdot 0^3$$

La desigualdad se cumple. El término se puede acotar con $k_2 = c_1$ y $n_0 = 0$, para $n_0 \geq 0$.

- Tercer término: c_3
Sabemos que $1 \leq n^3$ Si multiplicamos a ambos lados por c_3 ,

$$c_3 n \leq c_3 n^3$$

Encontramos un valor $k_3 = c_3$. Ahora, si probamos con $n_0 = 1$.

$$c_3 \leq c_3 \cdot 1^3$$

La desigualdad se cumple. El término se puede acotar con $k_3 = c_3$ y $n_0 = 1$, para $n_0 \geq 1$.

Ahora, sumamos todos los resultados parciales que obtuvimos antes.

$$c_2 n^3 + c_1 n^2 + c_3 \leq k_1 n^3 + k_2 n^3 + k_3 n^3$$

$$T(n) \leq (k_1 + k_2 + k_3) n^3$$

$$T(n) \leq (c_1 + c_2 + c_3) n^3$$

Renombrando $c = c_1 + c_2 + c_3$

$$T(n) \leq c n^3$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(n^3), \text{ con } c = c_1 + c_2 + c_3 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$

16 Ejercicio 18

Dado el siguiente algoritmo exprese, desarrolle e indique el orden de la función de tiempo de ejecución $T(n)$:

```
public static void metodo(int n){
    int i = 1;
    int j = n;
    while (i <= n){
        while (j > 1){
```



```

        for (int k = i; k < i+10; k++){
            System.out.println(k*i*j);
        }
        j = j / 2;
    }
    i = i + 2;
}
}

```

En primer lugar tenemos dos operaciones constantes que unificamos y llamamos c_1 . Luego, tenemos 3 iteraciones. Analizamos cuantas veces se ejecutan:

- while($i=j$)
 1. $i = 1$
 2. $i = 1 + 2$
 3. $i = 1 + 2 + 2 = 1 + 2 \cdot 2$
 4. $i = 1 + 2 \cdot 2 + 2 = 1 + 2 \cdot 3$
 - $\vdots$

Paso h : $i = 1 + 2(h - 1)$

Entonces, cuando :

$$\begin{aligned}
 i &= n \\
 n &= 1 + 2(h - 1) \\
 n - 1 &= 2h - 2 \\
 \frac{n + 1}{2} &= h
 \end{aligned}$$

La primera iteración se ejecuta $\frac{n+1}{2}$ veces.

- while($j \geq 1$) Como j no se inicializa en cada iteración del primer while, sólo se va a ejecutar cuando $i = 1$. Veamos los valores de j .

1. $j = 1$
2. $j = \frac{n}{2}$
3. $j = \frac{n}{2 \cdot 2} = \frac{n}{2^2}$
4. $j = \frac{n}{2^2 \cdot 2} = \frac{n}{2^3}$
- $\vdots$

Paso f : $j = \frac{n}{2^{f-1}}$

Cuando

$$\begin{aligned}j &= 1 \\1 &= \frac{n}{2^{f-1}} \\2^{f-1} &= n \\f &= 1 + \log_2 n\end{aligned}$$

Cuando $j = 1$, $f = 1 + \log_2 n$ pero como la condición de corte es $j > 1$, la iteración se ejecuta $\log_2 n$ veces.

- for (int k = i; k ≤ i+10; k++) Como k aumenta de a 1, esta iteración se ejecuta 10 veces en cada iteración de j .

Entonces tenemos

- $c_1 \rightarrow$ Operaciones constantes del inicio
- $\sum_{k=1}^{10} c_2 \rightarrow$ Ejecuciones del for (tercera iteración)
- $\sum_{j=1}^{\log_2 n} [\sum_{k=1}^{10} c_2] \rightarrow$ Ejecuciones del while interno (segunda iteración). Este término corresponde a la primera iteración del primer while, cuando $i = 1$.
- $\sum_{i=2}^{\frac{n+1}{2}} c_3 \rightarrow$ Ejecuciones restantes del primer while. En estos casos no se ejecutan las iteraciones internas porque $j = 1$, no entra en el segundo while.

Tenemos

$$\begin{aligned}T(n) &= c_1 + \sum_{j=1}^{\log_2 n} [\sum_{k=1}^{10} c_2] + \sum_{i=2}^{\frac{n+1}{2}} c_3 \\&= c_1 + \sum_{j=1}^{\log_2 n} c_2(10 + 1 - 1) + c_3 \sum_{i=2}^{\frac{n+1}{2}} 1 \\&= c_1 + 10c_2 \sum_{j=1}^{\log_2 n} 1 + c_3 \left(\frac{n+1}{2} - 2 + 1 \right) \\&= c_1 + 10c_2(\log_2 n + 1 - 1) + c_3 \left(\frac{n+1}{2} \right) \\&= c_1 + \frac{c_3}{2} + 10c_2 \log_2 n + \frac{c_3}{2} \\ \text{Renombramos } c_4 &= c_1 + \frac{c_3}{2}, c_5 = 10c_2 \text{ y } c_6 = \frac{c_3}{2} \\&= c_4 + c_5 \log_2 n + c_6 n\end{aligned}$$

Entonces, la función de tiempo de ejecución del método es

$$\boxed{T(n) = c_4 + c_5 \log_2 n + c_6 n}$$

Vamos a demostrar el orden de la función. Hay que demostrar que

$$T(n) = c_4 + c_5 \log_2 n + c_6 n \leq O(n)$$

Hay que buscar los valores constantes c y n_0 tales que

$$c_4 + c_5 \log_2 n + c_6 n \leq c \cdot n, \text{ para todo } n \geq n_0$$

Se tiene que demostrar que

$$T(n) = c_4 + c_5 \log_2 n + c_6 n \leq O(n)$$

Para esto, tenemos que encontrar una constante $c > 0$ y un n_0 tales que

$$c_4 + c_5 \log_2 n + c_6 n \leq cn \text{ para todo } n \geq n_0$$

Analizamos término a término,

- Primer término, c_4
Sabemos que $1 \leq n$. Si multiplicamos a ambos lados por c_4 .

$$c_4 \leq c_4 n$$

Encontramos un valor $k_1 = c_4$. Ahora, si probamos con $n_0 = 1$.

$$c_4 \leq c_4 \cdot 1$$

La desigualdad se cumple. El término se puede acotar con $k_1 = c_4$ y $n_0 = 1$, para $n_0 \geq 1$.

- Segundo término, $c_5 \log_2 n$
Sabemos que $\log_2 n \leq n$. Si multiplicamos a ambos lados por c_5 .

$$c_5 \log_2 n \leq c_5 n$$

Encontramos un valor $k_2 = c_5$. Ahora, si probamos con $n_0 = 1$.

$$c_5 \log_2 1 \leq c_5 \cdot 1$$

$$c_5 \cdot 0 \leq c_5$$

$$0 \leq c_5$$

La desigualdad se cumple porque $c_5 \geq 0$. Los valores que acotan al término son $k_2 = c_5$ y $n_0 = 1$.

- Tercer término, $c_6 n$ Sabemos que $n \leq n$. Si multiplicamos a ambos lados por c_6 .

$$c_6 n \leq c_6 n$$

Encontramos un valor $k_3 = c_6$. Ahora, si probamos con $n_0 = 0$.

$$\begin{aligned} c_6 \cdot 0 &\leq c_6 \cdot 0 \\ 0 &\leq 0 \end{aligned}$$

La desigualdad se cumple. El término se puede acotar con $k_3 = c_6$ y $n_0 = 0$, para $n_0 \geq 0$.

Ahora, sumamos todos los resultados parciales que obtuvimos antes.

$$\begin{aligned} c_4 + c_5 \log_2 n + c_6 n &\leq k_1 n + k_2 n + k_3 n \\ T(n) &\leq (k_1 + k_2 + k_3)n \\ T(n) &\leq (c_4 + c_5 + c_6)n \\ \text{Renombramos } c &= c_4 + c_5 + c_6 \\ T(n) &\leq cn \end{aligned}$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(n), \text{ con } c = c_4 + c_5 + c_6 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$

Nota: Puede haber algún detalle o error que se me haya escapado. No me juzguen, sólo soy una chica.

